ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



BÁO CÁO
ĐỒ ÁN TỐT NGHIỆP

Hiện thực bộ tăng tốc CNN dựa trên RISC-V

Ngành: KỸ THUẬT MÁY TÍNH

HỘI ĐỒNG: **Hội đồng 2 Kỹ thuật Máy tính**

GVHD: **PGS.TS Trần Ngọc Thịnh**

ThS. Huỳnh Phúc Nghị

GVPB: **ThS. Phạm Kiều Nhật Anh**

—o0o—

SVTH: **Trần Đức Mạnh - 2114026**

TP. HỒ CHÍ MINH, 10/2025

Chương 1

THIẾT KẾ BỘ TĂNG TỐC

1.1 Giới thiệu

Bộ tăng tốc - hay gọi là Flash Accelerator được thiết kế để tăng tốc quá trình tính toán của toàn hệ thống nói chung và mô hình Deep Learning nói riêng. Bộ tăng tốc Flash Accelerator hỗ trợ các tính năng sau:

- Hỗ trợ tính toán các Layer chính của mạng CNN:
  - Convoltional Layer
  - Max Pooling Layer
  - Fully-Connected Layer
- Flash Accelerator hiện tại hỗ trợ tính toán Activation Function thông qua hai loại hàm là ReLU và ReLU6.
- Trong thiết kế bộ tăng tốc CNN, Flash Accelerator được hỗ trợ ba chế độ xử lý tính toán chính, được tổ chức theo chu trình pipeline: **Convolution Layer**, **Mixed Layer** và **Fully-Connected Layer**. Việc phân tách này xuất phát từ đặc trưng của các kiến trúc CNN truyền thống cũng như những biến thể hiện đại, trong đó mỗi nhóm lớp có đặc điểm tính toán và yêu cầu bộ nhớ khác nhau. Cụ thể về các Layer như sau:
  - **Convolution Layer:** Thực hiện các phép toán Convolution, vốn chiếm phần lớn chi phí tính toán trong các mô hình như LeNet, VGG hay ResNet.
  - **Mixed Layer:** Bộ tăng tốc xử lý chuỗi tác vụ gồm Convolutional Layer kết hợp với Max-Pooling Layer. Hầu hết các kiến trúc CNN phổ biến, đặc biệt là các thế hệ đầu như LeNet-5, AlexNet, hay VGG, các lớp pooling hầu như luôn theo sau một lớp convolution để giảm chiều dữ liệu và tránh hiện tượng quá khớp (overfitting). Do đó, việc gom hai lớp này vào một chu trình tính toán không chỉ giảm overhead truyền dữ liệu mà còn tận dụng pipeline hiệu quả hơn.
  - **Fully-Connected Layer:** Bộ tăng tốc đảm nhận việc xử lý các phép tính của lớp Fully-Connected. Đây thường là lớp cuối cùng trong mạng CNN trước giai đoạn xử lý kết quả (ví dụ: Softmax hoặc Argmax để đưa ra phân bố xác suất).
- Bộ tăng tốc hỗ trợ hoàn toàn các mô hình CNN rút gọn dựa trên 8-bit Integer Quantization, nhằm đảm bảo hiệu quả cả về tính toán lẫn truy xuất dữ liệu.

- Flash Accelerator được thiết kế với cơ chế pipeline gồm bốn giai đoạn: RDATA (đọc dữ liệu), COMPS (tính toán), POOL (thực hiện Max Pooling) và WBACK (ghi kết quả).

1.2 Kiến trúc bộ tăng tốc

1.2.1 Định nghĩa phạm vi dữ liệu đầu vào

Để mô tả khả năng xử lý của bộ tăng tốc Flash, trước tiên cần đưa ra các giới hạn kích thước của dữ liệu đầu vào. Các giới hạn này được quy ước theo các tham số sau:

- **Tham số hình học:**
  - L_w: độ rộng tối đa được phép của *Input Feature Map* và *Kernel*.
  - L_h: chiều cao tối đa được phép của *Input Feature Map* và *Kernel*.
  - L_c: số kênh (chiều sâu) tối đa được phép của *Input Feature Map* và *Kernel*.
  - L_k: số lượng tối đa của *Kernel* trong Convolution/Mixed-Layer hoặc số lượng *Weight* trong Fully-Connected Layer. Đây cũng là số phần tử *Bias* cần thiết trong một chu trình tính toán.
- **Số lượng phần tử dữ liệu:**
  - Số phần tử đầu vào tối đa (Input Feature Map hoặc Input Activation):
$$L_w \times L_h \times L_c$$
  - Số lượng trọng số tối đa (Kernel hoặc Weight):
$$L_w \times L_h \times L_c \times L_k$$
  - Số lượng *Bias* cực đại:
$$L_k$$

Dựa trên các tham số trên, phạm vi tính toán có thể được đặc tả cho từng loại lớp như sau:

- **Convolutional Layer**: Đầu vào là một *Input Feature Map* có kích thước:
$$L_w \times L_h \times L_c,$$
và mỗi *Kernel* có kích thước:
$$L_w \times L_h \times L_c.$$
Với tổng cộng L_k kernel, số lượng trọng số cần thiết là:
$$L_w \times L_h \times L_c \times L_k.$$
- **Fully-Connected Layer**: Đầu vào 3D sẽ được *flatten* về dạng vector 1D có độ dài:
$$\underbrace{L_w \times L_h \times L_c}_{\text{width}},$$
và trọng số có kích thước:
$$\underbrace{L_w \times L_h \times L_c}_{\text{width}} \times L_k.$$

Như vậy, mỗi lớp Fully-Connected tạo ra L_k đầu ra ứng với L_k neuron.

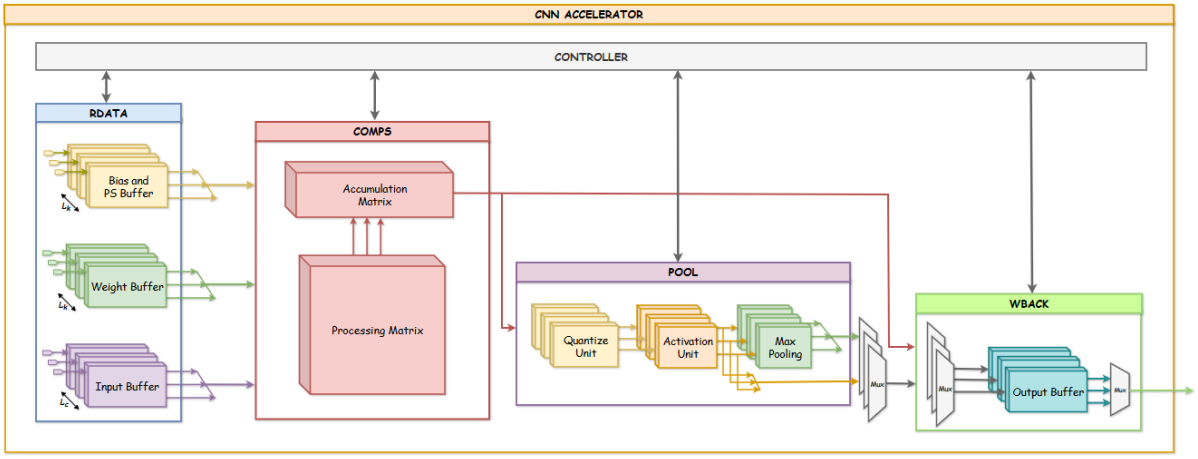
Trong phiên bản triển khai hiện tại, các tham số mặc định được lựa chọn là:

$$(L_w, L_h, L_c, L_k) = (3, 3, 3, 3),$$

nhằm phục vụ cho việc kiểm chứng và đánh giá khả năng hoạt động của bộ tăng tốc trước khi mở rộng sang các cấu hình lớn hơn.

1.2.2 Kiến trúc tổng quát

Kiến trúc tổng quát của Flash Accelerator được thể hiện ở Hình 1.1:



Hình 1.1: *Sơ đồ khối về bộ tăng tốc Flash*

Kiến trúc tổng quát của Flash Accelerator bao gồm bốn khối chức năng chính và một khối điều khiển trung tâm, được mô tả như sau:

- **Controller**: Đây là khối điều khiển trung tâm, có nhiệm vụ tiếp nhận *instruction* và các thông số cấu hình của từng lớp đầu vào. Dựa trên thông tin này, Controller xác định chế độ hoạt động (Convolution, Mixed hoặc Fully-Connected Layer), đồng thời điều phối các nguồn dữ liệu qua các giai đoạn RDATA, COMPS, POOL và WBACK. Bên cạnh đó, Controller quản lý và đồng bộ trạng thái của toàn bộ pipeline, bảo đảm các khối chức năng vận hành tuần tự và chính xác trong từng chu kỳ xử lý.

- **RDATA**: Đây là nhóm chức năng thực hiện việc chuẩn bị và cung cấp dữ liệu đầu vào cho khối tính toán. Trong thiết kế bộ tăng tốc, RDATA bao gồm ba loại bộ nhớ đệm chính, mỗi loại đảm nhận một vai trò cụ thể trong quá trình tính toán:

  – *Bias and Partial-Sum Buffer*: Bộ nhớ đệm dùng để lưu trữ các giá trị *Bias* cũng như các *Partial-Sum*. Bộ nhớ này lưu trữ các giá trị *Bias* và các *Partial-Sum* (tổng trung gian). Khi lớp mạng có kích thước vượt quá khả năng xử lý trong một chu kỳ, các phép tích chập hoặc nhân ma trận sẽ được chia thành nhiều bước. Kết quả của mỗi bước được lưu vào *Partial-Sum Buffer* để tái sử dụng ở các chu kỳ tiếp theo, đảm bảo tính đúng đắn của kết quả cuối cùng. Trong cấu hình mặc định của Flash Accelerator, số lượng *Bias and Partial-Sum Buffer* là L_k.

  – *Weight Buffer*: Đây là vùng lưu trữ tạm thời cho các tham số huấn luyện của mô hình, bao gồm *Kernel* trong Convolution Layer hoặc *Weight* trong Fully-Connected Layer.
  Trong cấu hình mặc định của Flash Accelerator, số lượng Weight Buffer là L_k.

  - *Input Buffer*: Bộ nhớ đệm dành cho dữ liệu đầu vào của lớp, có thể là *Input Activation* (trong Fully-Connected Layer) hoặc *Input Feature Map* (trong Convolution Layer).
    Trong cấu hình mặc định của Flash Accelerator, số lượng Input Buffer là L_c.

  Các dữ liệu này sẽ được cung cấp tuần tự cho khối COMPS theo đúng thứ tự địa chỉ được định nghĩa.

- **COMPS**: Đây là nhóm chức năng đảm nhận toàn bộ tác vụ tính toán của bộ tăng tốc. Khối quan trọng nhất của COMPS là *Processing Matrix*, một mảng phần tử xử lý song song được thiết kế để thực hiện các phép *Multiply-Accumulate* (MAC). Các phép MAC này có thể sinh ra *Partial-Sum* hoặc *Final-Sum*, tùy thuộc vào loại lớp được xử lý và được *Accumulation Matrix* để cộng dồn kết quả.

  - *Đối với Convolution Layer*: COMPS khai thác cấu trúc ba chiều của dữ liệu, bao gồm (L_w, L_h, L_c) cho *Input Feature Map* và (L_w, L_h, L_c, L_k) cho *Kernel*. Việc triển khai tính toán theo không gian 3D cho phép hệ thống thực hiện phép tích chập một cách song song, giảm đáng kể số lượng *Partial-Sum* cần lưu trữ và tái sử dụng. Nhờ đó, chi phí truy xuất bộ nhớ trung gian được giảm thiểu và hiệu năng tổng thể của pipeline được cải thiện đáng kể.
  - *Đối với Fully-Connected Layer*: Tác vụ chính là phép nhân–cộng giữa ma trận trọng số và vector đầu vào trong không gian hai chiều. Đây là phép tính trực tiếp và ít phức tạp hơn so với Convolution, nhưng vẫn được xử lý trong cùng *Processing Matrix* để duy trì tính thống nhất kiến trúc.

- **POOL**: Đây là khối hậu xử lý, chỉ được kích hoạt sau khi COMPS hoàn tất tính toán và tạo ra tổng cuối cùng. POOL có ba đơn vị chức năng chính:

  - *Quantize Unit*: Thực hiện việc làm tròn và đưa kết quả về dạng số nguyên 8-bit. Cụ thể, mỗi giá trị đầu ra từ khối *COMPS* có độ dài 32-bit sẽ được nhân với tham số lượng tử và sau đó thu gọn về định dạng 8-bit, nhằm giảm chi phí lưu trữ và băng thông truyền dữ liệu.
  - *Activation Unit*: Đảm nhiệm việc áp dụng các hàm kích hoạt phi tuyến. Trong thiết kế hiện tại, hai hàm kích hoạt phổ biến được hỗ trợ là *ReLU* và *ReLU6*.
  - *Compare Unit*: Thực hiện thao tác so sánh giá trị, được sử dụng chủ yếu trong các bước Max-Pooling. Khối Compare Unit chỉ xuất hiện trong *Mixed Layer*, và cơ chế hoạt động chi tiết sẽ được trình bày ở phần thiết kế chức năng chuyên biệt.

- **WBACK**: Đây là nhóm chức năng đảm nhiệm tác vụ ghi dữ liệu, nằm ở giai đoạn cuối cùng của pipeline. Nhóm này bao gồm hai thành phần chính:

  - *Output Buffer*: Bộ nhớ đệm dung lượng nhỏ, dùng để lưu trữ tạm thời các giá trị đầu ra của Layer, có thể là *Partial-Sum* hoặc kết quả hoàn chỉnh sau khi đã qua các phép xử lý ở khối POOL (ví dụ: lượng tử hóa, kích hoạt).
  - *Output Buffer Mux*: Khối chọn dữ liệu đầu ra, cho phép lựa chọn luồng dữ liệu được ghi vào Output Buffer. Cụ thể, kết quả có thể được lấy trực tiếp từ COMPS (trong trường hợp chỉ cần tổng riêng phần) hoặc từ POOL (trong trường hợp đã hoàn tất các phép hậu xử lý).

1.2.3 Thiết kế các khối chức năng

1.2.3.1 Nhóm thực hiện tác vụ đọc dữ liệu (RDATA)

Nhóm chức năng này đảm nhiệm vai trò tiếp nhận và lưu trữ dữ liệu từ bộ nhớ ngoài (Data Memory) để cung cấp cho pipeline tính toán. Cụ thể, nhóm bao gồm ba loại bộ nhớ chính như sau:

A Bias and Partial-Sum Buffer.

Đây là bộ nhớ đệm dành cho các giá trị *Bias* và *Partial-Sum* (tổng riêng phần). Trong mỗi chu kỳ, bộ tăng tốc chỉ cần nạp một đơn vị dữ liệu 32-bit từ bộ nhớ ngoài để lưu trữ vào buffer, do tần suất xuất hiện của các giá trị này tương đối thấp. Các *Partial-Sum* được sử dụng trong trường hợp một lớp có kích thước lớn, khiến việc tính toán phải chia thành nhiều bước. Kết quả trung gian của mỗi bước được ghi lại và tái sử dụng ở các chu kỳ tiếp theo cho đến khi tổng hợp thành kết quả cuối cùng. Tổng số buffer loại này tỉ lệ thuận với L_k, cho phép duy trì đủ số lượng *Bias*/*Partial-Sum* cần thiết cho mỗi chu kỳ tính toán.

B Weight Buffer.

Bộ nhớ đệm này lưu trữ các tham số của mô hình, cụ thể là *Weight* trong Fully-Connected Layer hoặc *Kernel* trong Convolution Layer. Mỗi buffer có dung lượng đủ để chứa $L_w \times L_h$ giá trị 8-bit, đảm bảo cung cấp cho toàn bộ phép nhân–cộng của một ô tính toán trong chu kỳ hiện tại. Do mỗi lần truy xuất bộ nhớ ngoài có độ rộng 32-bit, lớn gấp bốn lần so với kích thước một trọng số (8-bit), số lần đọc cần thiết được giảm xuống còn:

$$\left\lceil \frac{L_w \times L_h}{4} \right\rceil.$$

Trong toàn hệ thống, tổng cộng có L_k khối Weight Buffer, đảm bảo duy trì đồng thời $L_w \times L_h \times L_k$ trọng số cho các phép tính song song.

C Input Buffer.

Đây là bộ nhớ đệm dành cho dữ liệu đầu vào, có thể là *Input Activation* (trong Fully-Connected Layer) hoặc *Input Feature Map* (trong Convolution Layer). Dung lượng mỗi buffer được ánh xạ theo ba chiều (L_w, L_h, L_c), với kích thước cơ bản $L_w \times L_h$ giá trị 8-bit. Đặc tính cục bộ của dữ liệu trong Convolution cho phép tải thêm nhiều phần tử trong cùng một hàng, sau đó tái sử dụng ở các chu kỳ tiếp theo. Nhờ đó, số lần đọc từ bộ nhớ ngoài giảm xuống còn:

$$L_w \times \left\lceil \frac{L_h}{4} \right\rceil$$

cho mỗi Input Buffer. Ngược lại, trong Fully-Connected Layer, dữ liệu được lưu trữ liên tục nên số lần đọc tối thiểu là:

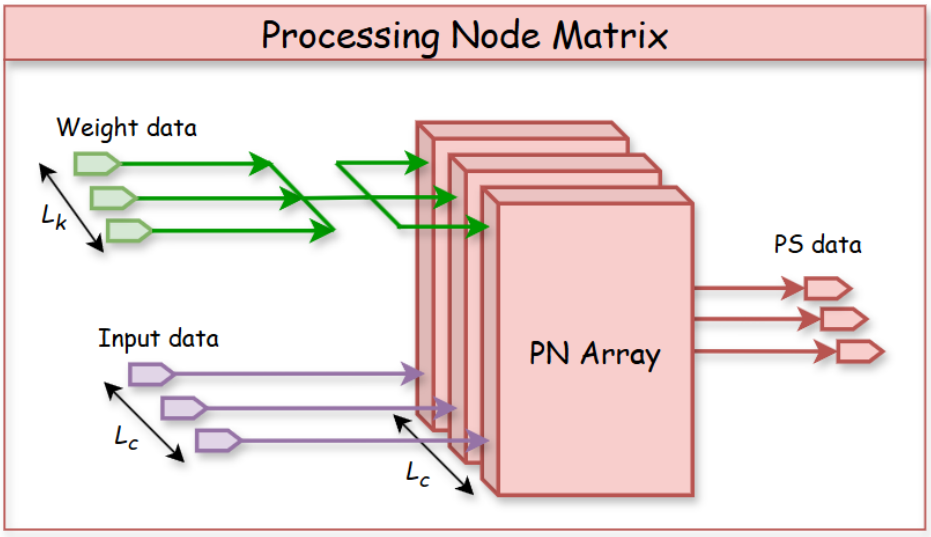
$$\left\lceil \frac{L_w \times L_h}{4} \right\rceil.$$

Tổng cộng hệ thống có L_c Input Buffer, cho phép duy trì ít nhất $L_w \times L_h \times L_c$ giá trị đầu vào tại chỗ, đủ để phục vụ cho một chu kỳ tính toán. Việc tái sử dụng dữ liệu này không chỉ giảm số lần đọc từ bộ nhớ ngoài mà còn giúp cải thiện đáng kể băng thông hiệu dụng của pipeline.

1.2.3.2 Nhóm thực hiện tác vụ tính toán (COMPS)

A Processing Matrix.

Đây là khối chức năng trung tâm, đảm nhận toàn bộ tác vụ tính toán của bộ tăng tốc. Yêu cầu quan trọng nhất của Processing Matrix là phải thực hiện các phép toán một cách song song và hiệu quả, vì đây là nơi quyết định trực tiếp đến thông lượng của toàn bộ hệ thống.
Kiến trúc tổng quát của Processing Matrix được thể hiện ở Hình 1.2:



Hình 1.2: *Sơ đồ khối của Processing Matrix*

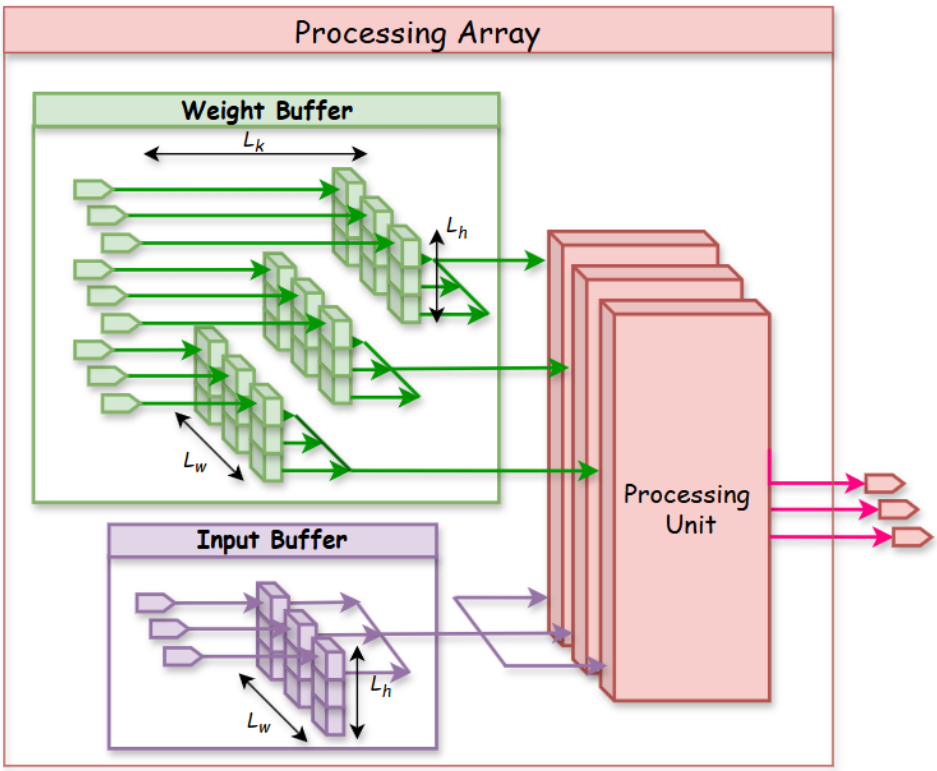
Trong *Convolution Layer*, Processing Matrix tận dụng đặc tính ba chiều của dữ liệu, bao gồm *Input Feature Map* với kích thước (L_w, L_h, L_c) và *Kernel* với kích thước (L_w, L_h, L_c, L_k). Nhờ đó, các phép tích chập có thể được thực hiện đồng thời trên ba chiều dữ liệu, giúp giảm đáng kể số lượng *Partial-Sum* cần lưu trữ và tái sử dụng.

Trong *Fully-Connected Layer*, phép tính chỉ diễn ra trong không gian hai chiều, do đó không cần khai thác toàn bộ cấu trúc 3D của Processing Matrix. Ở trường hợp này, chỉ một *Processing Array* duy nhất được thực hiện phép nhân–cộng giữa $\underbrace{L_w \times L_h \times L_c}_{\text{width}}$ đầu vào và $\underbrace{L_w \times L_h \times L_c}_{\text{width}} \times L_k$ trọng số tương ứng.

a) Processing Array

Khối Processing Array đảm nhận việc thực hiện các phép nhân–cộng (MAC) trong không gian hai chiều (2D). Điểm đặc trưng của khối này là có khả năng xử lý đồng thời với L_k trọng số khác nhau. Nói cách khác, với một dữ liệu đầu vào duy nhất, Processing Array sẽ thực hiện phép tích chập song song với L_k bộ *Weight*.

Kiến trúc tổng quát của Processing Array được thể hiện ở Hình 1.3:



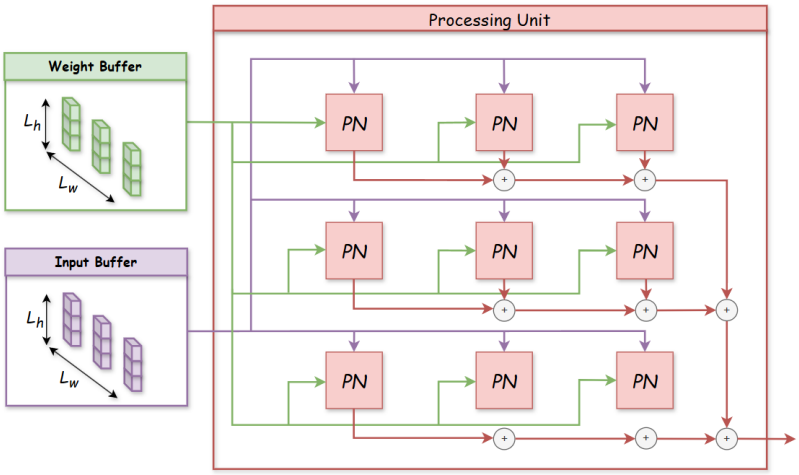
Hình 1.3: *Sơ đồ khối của Processing Array*

Khối Processing Array bao gồm:

- *Processing Unit*: Đây là phần tử tính toán cơ bản nhất. Processing Array có tổng cộng L_k Processing Unit, mỗi đơn vị đảm nhận một phép tính 2D tương ứng với một bộ trọng số (*Weight*). Nhờ đó, cùng một dữ liệu đầu vào có thể được nhân–cộng song song với L_k bộ trọng số khác nhau.
- *Weight Register*: Các thanh ghi này lưu trữ trọng số cần thiết cho quá trình tính toán. Mỗi Processing Unit gắn với một nhóm Weight Register riêng, vì vậy toàn bộ Processing Array cần tổng cộng L_k bộ trọng số. Mỗi bộ trọng số có kích thước $L_w \times L_h$, được sử dụng để thực hiện phép tích chập.
- *Input Register*: Đây là nơi lưu trữ dữ liệu đầu vào (Input Feature Map trong Convolution/Mixed Layer mode hoặc Input Flatten trong Fully-Connected mode). Tất cả L_k Processing Unit cùng chia sẻ chung một Input Register.

b) Processing Unit

Processing Unit (PU) là khối tính toán trung gian trong Processing Array, chịu trách nhiệm thực hiện một phép nhân–cộng (MAC) đầy đủ trên một khối dữ liệu đầu vào $L_w \times L_h$ với trọng số cùng kích thước. Kiến trúc tổng quát của Processing Unit được thể hiện ở Hình 1.4:



Hình 1.4: *Sơ đồ khối của Processing Unit*

Một khối PU bao gồm ba thành phần chính:

- *Input Buffer*: Lưu trữ một khối dữ liệu đầu vào có kích thước $L_w \times L_h$ (8-bit), tương ứng với một vùng trong *Input Feature Map* hoặc một đoạn trong *Input Flatten*.
- *Weight Buffer*: Lưu trữ tập trọng số có kích thước $L_w \times L_h$, đóng vai trò như một *Kernel* trong Convolution Layer hoặc một vector trọng số trong Fully-Connected Layer.
- *Processing Node*: Đây là mảng tính toán gồm $3 \times 3 = 9$ *Processing Node* (PN), được bố trí thành ma trận 2D. Mỗi PN đảm nhận một phép nhân–cộng cơ bản giữa một phần tử đầu vào và trọng số tương ứng. Các kết quả trung gian sau đó được cộng dồn lại để sinh ra một đầu ra duy nhất của PU.

Về mặt toán học, hoạt động của PU được mô tả như:

$$O = \sum_{i=1}^{L_h} \sum_{j=1}^{L_w} in_{i,j} \times w_{i,j},$$

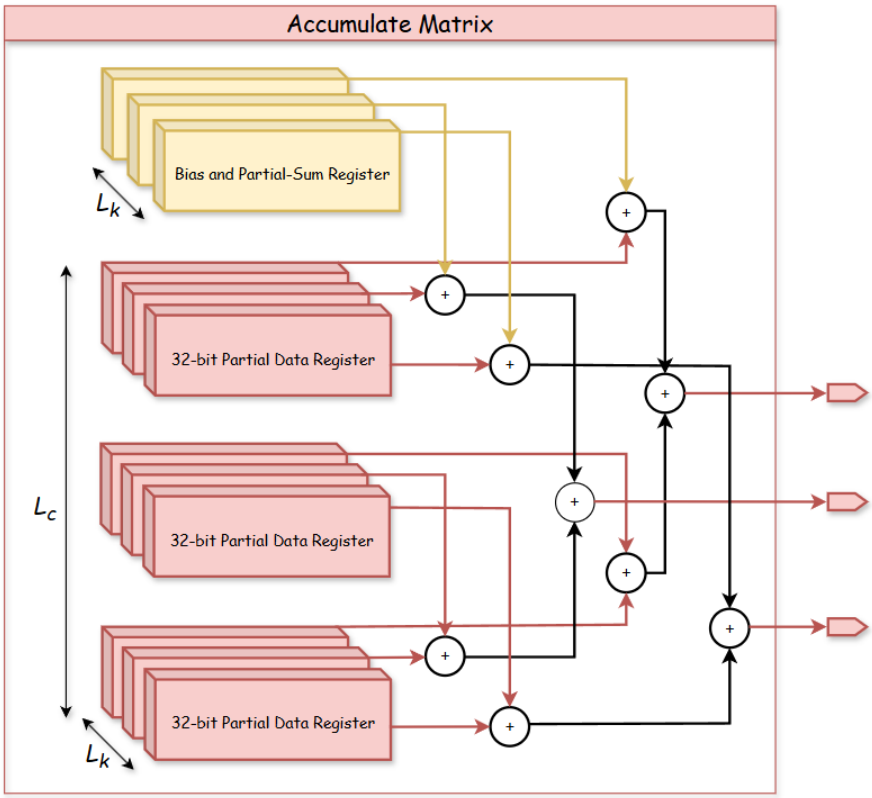
trong đó $in_{i,j}$ là dữ liệu đầu vào tại vị trí (i, j) và $w_{i,j}$ là trọng số tương ứng.
Mỗi Processing Unit bao gồm nhiều *Processing Node* (PN), trong đó mỗi PN đảm nhận một phép nhân cơ bản giữa một phần tử đầu vào $in_{i,j}$ và một trọng số $w_{i,j}$. Các PN hoạt động song song, sinh ra nhiều kết quả nhân trong cùng một chu kỳ, trước khi được cộng dồn lại để tạo thành giá trị đầu ra của PU.
Để tối ưu hiệu năng, mỗi PN được ánh xạ trực tiếp xuống một *DSP Slice* trên FPGA. DSP Slice là tài nguyên phần cứng chuyên dụng cho các phép toán số học, có khả năng thực hiện phép nhân–cộng (MAC) ở tốc độ cao và độ chính xác ổn định. Việc sử dụng DSP mang lại các lợi ích sau:

- **Hiệu năng**: đảm bảo thực hiện nhiều phép nhân song song trong một chu kỳ xung nhịp.
- **Độ trễ thấp**: giảm số tầng logic so với triển khai bằng LUT thông thường.
- **Tiết kiệm tài nguyên**: giảm thiểu việc sử dụng LUT/FF, dành tài nguyên này cho các khối điều khiển và logic phụ trợ.

Nhờ tổ chức thành nhiều PN song song và tận dụng triệt để DSP Slice, PU có thể đạt được hiệu năng tính toán tối ưu, phù hợp cho cả các phép tích chập trong Convolution Layer lẫn nhân ma trận–vector trong Fully-Connected Layer.

B Accumulate Matrix.

Accumulate Matrix là khối thực hiện tích lũy và cộng dồn các tổng riêng phần được tạo ra từ Processing Matrix với giá trị Bias. Đầu vào của khối này là $L_k \times L_c$ giá trị dữ liệu, trong đó L_k là số lượng trọng số (hoặc kernel) và L_c là chiều sâu dữ liệu cần xử lý.
Kiến trúc tổng quát của Accumulate Matrix được thể hiện ở Hình 3.6:



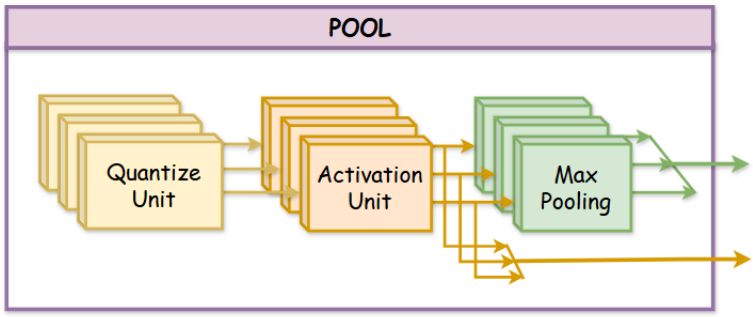
Hình 1.5: *Sơ đồ khối của Accumulate Matrix*

Khối Accumulate Matrix bao gồm ba thành phần chính:

- **Bias and Partial-Sum Register**: Đây là bộ thanh ghi có nhiệm vụ lưu trữ giá trị bias và các tổng riêng phần (partial sum) được tạo ra từ những chu kỳ tính toán trước.
- **Intermediate Register**: Các thanh ghi trung gian này được sử dụng để lưu trữ kết quả tính toán tạm thời các tổng riêng phần được tính toán bởi Processing Matrix
- **Adder**: Bộ cộng thực hiện phép cộng giữa dữ liệu đầu ra tức thời từ Processing Matrix và giá trị tổng riêng phần hoặc giá trị Bias được lưu trong các thanh ghi. Quá trình cộng tích lũy các giá trị tổng riêng phần dược tính toán theo mô hình cây nhị phân Adder Tree.

1.2.3.3 Nhóm thực hiện tác vụ hậu xử lý (POOL)

POOL là khối đảm nhiệm các chức năng hậu xử lý sau khi đã thu được kết quả từ Accumulate Matrix. Trong pipeline của hệ thống, POOL chỉ thực sự được kích hoạt khi dữ liệu đầu vào là *tổng cuối cùng*. Trong các trường hợp dữ liệu vẫn ở dạng *tổng riêng phần*, POOL chỉ hoạt động như một bộ đệm trung gian, chuyển tiếp dữ liệu trực tiếp từ COMPS sang WBACK mà không thực hiện thêm bất kỳ phép xử lý nào.
Kiến trúc tổng quát của POOL được thể hiện ở Hình 1.6:



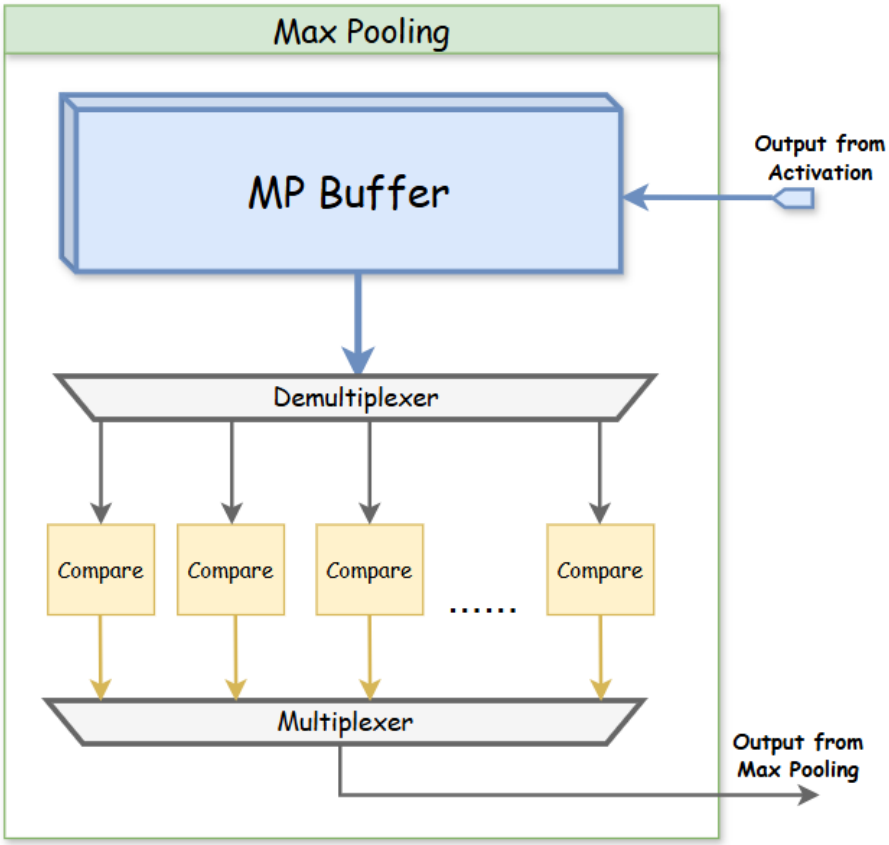
Hình 1.6: *Sơ đồ khối của POOL*

Khối POOL bao gồm ba thành phần chính:

- **Quantize Unit**: Thực hiện lượng tử hóa kết quả từ 32-bit về dạng số nguyên 8-bit để đảm bảo tính tương thích với toàn bộ pipeline. Quá trình này được hiện thực theo đặc tả: nhân kết quả với hệ số M_0, sau đó dịch phải n bit để tạo ra giá trị ước lượng gần nhất. Đây là bước cần thiết để duy trì tính chính xác khi làm việc với các phép toán dấu chấm động, đồng thời tối ưu việc lưu trữ và tính toán ở các tầng kế tiếp.
- **Activation Unit**: Áp dụng các hàm kích hoạt phi tuyến để tăng khả năng biểu diễn của mạng CNN. Trong thiết kế hiện tại, hai hàm được hỗ trợ là:
  - ReLU: $f(x) = \max(0, x)$
  - ReLU6: $f(x) = \min(\max(0, x), 6)$

  Các hàm kích hoạt này được lựa chọn vì tính đơn giản và hiệu quả trong việc loại bỏ các giá trị âm, đồng thời giới hạn miền giá trị đầu ra.
- **Max Pooling Unit**: Max Pooling là khối được sử dụng trong **Mixed Layer Mode**, khi cần kết hợp trực tiếp kết quả từ tầng Convolution với phép Pooling để giảm chiều dữ liệu. Mục tiêu của khối này là trích chọn giá trị lớn nhất trong một vùng cửa sổ (ví dụ 2×2 hoặc 3×3, hiện tại Max Pooling hỗ trợ 2×2), nhằm rút gọn thông tin mà vẫn giữ được đặc trưng quan trọng nhất.

Kiến trúc chi tiết của Max Pooling được thể hiện ở Hình 1.7:



Hình 1.7: *Sơ đồ kiến trúc chi tiết của Max Pooling*

Khối Max Pooling bao gồm bốn thành phần chính:

- **MP Buffer**: Đây là bộ nhớ đệm dùng để lưu trữ tạm thời dữ liệu đầu vào được gửi từ *Activation Unit*.
- **Demultiplexer (Demux)**: Chia dữ liệu từ MP Buffer thành nhiều luồng song song, sau đó phân phối chúng đến các khối Compare. Việc tách dữ liệu theo luồng cho phép khai thác tính song song trong quá trình tìm giá trị cực đại.
- **Compare Units**: Đây là các khối so sánh song song, mỗi khối nhận một phần dữ liệu từ Demux và thực hiện phép so sánh để xác định giá trị lớn nhất trong nhóm của mình. Cơ chế này tương đương với việc duy trì một thanh ghi tạm thời: nếu giá trị mới lớn hơn giá trị đang lưu, thanh ghi sẽ được cập nhật.
- **Multiplexer (Mux)**: Sau khi các Compare Units đưa ra kết quả, Multiplexer thực hiện chọn lọc và gộp các giá trị lớn nhất lại để cho ra kết quả cuối cùng của khối Max Pooling. Kết quả này sau đó sẽ được ghi vào *Output Buffer* để truyền sang giai đoạn WBACK.

1.2.3.4 Nhóm thực hiện tác vụ ghi dữ liệu (WBACK)

WBACK là giai đoạn cuối cùng trong pipeline, chịu trách nhiệm ghi dữ liệu từ bộ tăng tốc ra bộ nhớ ngoài. Dữ liệu đầu vào của WBACK có thể là *tổng riêng phần* 32-bit từ COMPS hoặc *kết quả cuối cùng* 8-bit từ POOL. Để thống nhất thiết kế, *Output Buffer* được xây dựng với độ rộng 32-bit, đủ để lưu trữ cả hai loại dữ liệu; trong trường hợp dữ liệu đầu ra chỉ có 8-bit thì 24-bit còn lại không được sử dụng. Kiến trúc WBACK bao gồm:

- **Input Multiplexer**: lựa chọn nguồn dữ liệu ghi vào Output Buffer, giữa kết quả từ COMPS (partial sum) hoặc từ POOL (final result).

- **Output Buffer**: bộ nhớ đệm tạm thời để lưu trữ kết quả trước khi đưa ra ngoài. Hệ thống có tổng cộng L_k Output Buffer, mỗi buffer tương ứng với một kênh kết quả độc lập.
- **Output Multiplexer**: lựa chọn dữ liệu từ Output Buffer để truyền ra bộ nhớ ngoài một cách tuần tự và đồng bộ.

Với cơ chế này, toàn bộ L_k kết quả song song sẽ được ghi ra ngoài trong L_k chu kỳ ghi, đảm bảo sự đồng bộ giữa pipeline nội bộ và hệ thống bộ nhớ ngoài.

1.3 Thiết kế điều khiển

Để Flash Accelerator vận hành theo đúng cơ chế pipeline và đảm bảo tính đồng bộ của dữ liệu tính toán, cần có một khối điều khiển trung tâm chịu trách nhiệm quản lý luồng thực thi của các khối chức năng, đồng thời cho phép tùy chỉnh hoạt động theo yêu cầu người dùng.

1.3.1 Cấu hình cho bộ tăng tốc

1.3.1.1 Thông tin của Layer

Mỗi Layer cần cung cấp đầy đủ các thông số cấu hình để bộ tăng tốc có thể tiến hành quá trình tính toán chính xác trên Layer đó. Các thông tin này có thể khác nhau tùy theo loại Layer, tuy nhiên mỗi loại đều yêu cầu tối thiểu các tham số cơ bản sau:

- **Fully-Connected Layer:**
  - Kích thước dữ liệu đầu vào (Input Activation): inW (1D)
  - Kích thước dữ liệu trọng số (Weight): $wW \times wH$ (2D)
  - Kích thước dữ liệu đầu ra (Output Activation): ouW (1D)
  - Kích thước dữ liệu đối số (Bias): bW (1D)
  - Loại hàm kích hoạt phi tuyến (Activation Function)
- **Convolution Layer:**
  - Kích thước bản đồ đặc trưng đầu vào (Input Feature Map): $ifmW \times ifmH \times ifmD$ (3D)
  - Kích thước dữ liệu trọng số (Kernel): $kW \times kH \times kD \times kN$ (4D)
  - Kích thước bản đồ đặc trưng đầu ra (Output Feature Map): $ofmW \times ofmH \times ofmD$ (3D)
  - Kích thước dữ liệu đối số (Bias): bW (1D)
  - Giá trị bước (Stride): sW (theo chiều ngang) và sH (theo chiều dọc)
  - Loại hàm kích hoạt phi tuyến (Activation Function)
- **Max Pooling Layer:**
  - Kích thước bản đồ đặc trưng đầu vào (Input Feature Map): $ifmW \times ifmH \times ifmD$ (3D)
  - Kích thước bản đồ đặc trưng đầu ra (Output Feature Map): $ofmW \times ofmH \times ofmD$ (3D)
  - Kích thước khối trượt (Patch Size): $PW \times PH$ (2D)
  - Giá trị bước (Stride): sW và sH

1.3.1.2 Thanh ghi cấu hình của Flash Accelerator

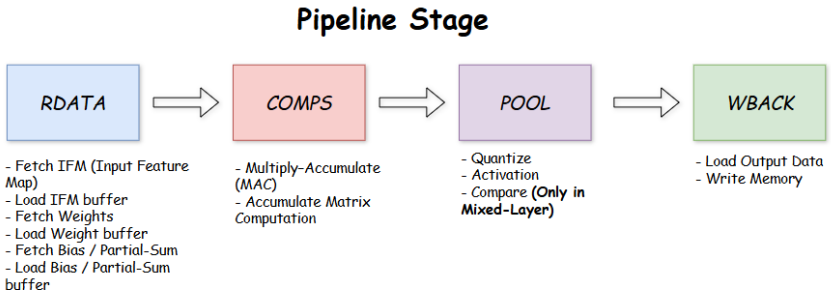
Bảng dưới đây mô tả chi tiết ý nghĩa và chức năng của các thanh ghi cấu hình trong khối điều khiển của Flash Accelerator. Các thanh ghi này cho phép CPU (PicoSoC) nạp các tham số điều khiển, địa chỉ vùng nhớ và hệ số lượng tử hóa trước khi khởi chạy tác vụ.

- **Thanh ghi 0**: Lưu trữ địa chỉ 32-bit cho Input Feature Map (IFM).
- **Thanh ghi 1**: Lưu trữ địa chỉ 32-bit cho Kernel (Weight).
- **Thanh ghi 2**: Lưu trữ địa chỉ 32-bit cho Output Feature Map (OFM).
- **Thanh ghi 3**: Lưu trữ địa chỉ 32-bit cho Bias.
- **Thanh ghi 4**: Lưu trữ địa chỉ 32-bit cho Partial-Sum.
- **Thanh ghi 5** (16-bit): Chứa các thông tin cấu hình:
  - [3:0] (4 bit): Loại layer (Convolution, Mixed, Fully).
  - [7:4] (4 bit): Loại hàm kích hoạt (ReLU, ReLU6).
  - [11:8] (4 bit): Stride width.
  - [15:12] (4 bit): Stride height.
- **Thanh ghi 6** (32-bit):
  - *[15:0]* (16 bit): Kernel patch width.
  - *[31:16]* (16 bit): Kernel patch height.
- **Thanh ghi 7** (32-bit):
  - [15:0] (16 bit): *nok_ ofm_ depth* – độ sâu của Output Feature Map.
  - [31:16] (16 bit): *kernel_ ifm_ depth* – độ sâu của Input Feature Map được kernel xử lý.
- **Thanh ghi 8** (32-bit):
  - [15:0] (16 bit): Input width.
  - [31:16] (16 bit): Input height.
- **Thanh ghi 9** (32-bit):
  - [15:0] (16 bit): Output width.
  - [31:16] (16 bit): Output height.
- **Thanh ghi 10** (32-bit):
  - [15:0] (16 bit): *input2D_ size*.
  - [31:16] (16 bit): *output2D_ size*.
- **Thanh ghi 11** (32-bit): *kernel3D_ size* – kích thước 3D của kernel.
- **Thanh ghi 12** (8-bit): *output_ quant_ buf_ insel* – bộ chọn kênh cho Output Quantize Buffer.
- **Thanh ghi 13** (32-bit): *output_ multiplier* – hệ số nhân dùng trong lượng tử hóa cho kênh đã chọn.
- **Thanh ghi 14** (8-bit): *output_ shift* – hệ số dịch bit dùng trong lượng tử hóa cho kênh đã chọn.

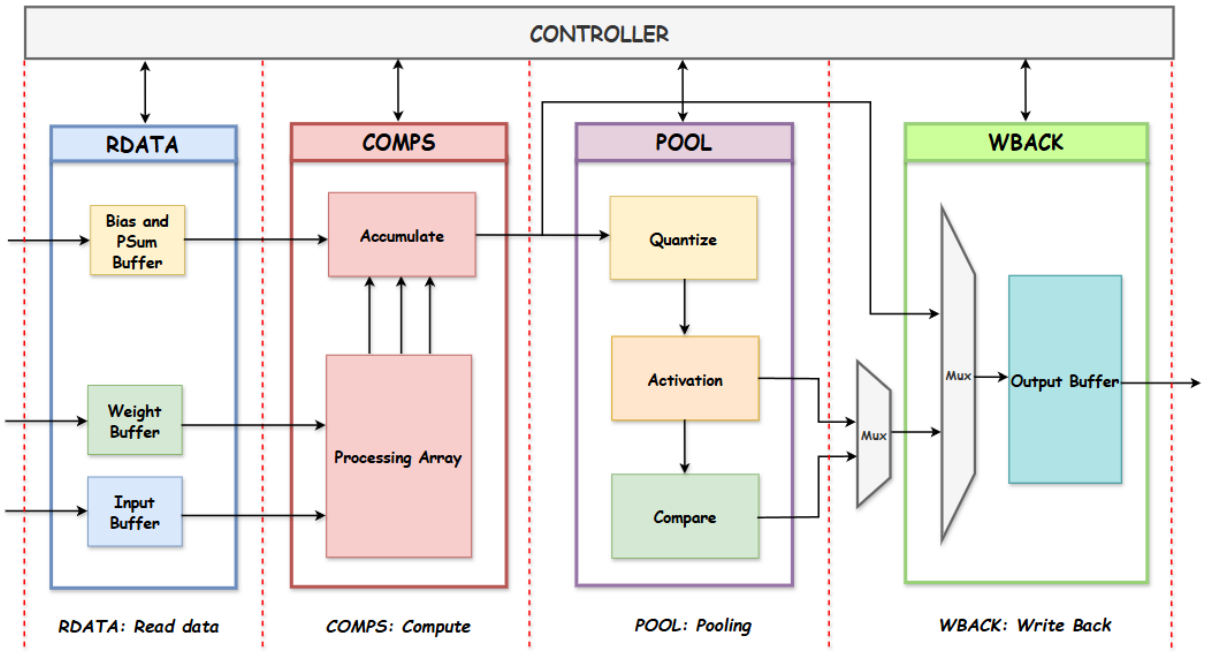
- **Thanh ghi 15** (32-bit): *input_ offset* – giá trị bù áp dụng cho dữ liệu đầu vào.
- **Thanh ghi 16** (32-bit): *output_ offset* – giá trị bù áp dụng cho dữ liệu đầu ra.
- **Thanh ghi 17** (32-bit):
  - [15:0] (16 bit): Output Pool width.
  - [31:16] (16 bit): Output Pool height.

  Thanh ghi này chỉ được sử dụng trong trường hợp Mixed Layer.
- **Thanh ghi 18** (32-bit): *output2D_ pool_ size* – kích thước lát cắt sau Pooling. Thanh ghi này chỉ được sử dụng trong trường hợp Mixed Layer.

1.3.2 Pipeline ở Flash Accelerator



Hình 1.8: *Pipeline của bộ tăng tốc Flash Accelerator*



Hình 1.9: *Phân chia tài nguyên Pipeline của bộ tăng tốc Flash Accelerator*

Flash Accelerator tiến hành xử lý tác vụ tính toán của từng mode theo cơ chế pipeline, trong đó số lượng stage sẽ thay đổi tùy thuộc vào loại mode. Cụ thể, với Mixed-Layer Mode, pipeline được chia thành 4 stage, trong khi với Convolution Layer Mode và Fully-Connected Layer, quá trình xử lý chỉ bao gồm 3 stage như mô tả dưới đây.
Đặc tả về từng giai đoạn trong Pipeline như ở dưới:

- **Read Data – RDATA**: Giai đoạn này có nhiệm vụ trích xuất dữ liệu cần thiết cho quá trình tính toán từ bộ nhớ ngoài và lưu trữ chúng vào các bộ nhớ đệm và thanh ghi nội bộ của bộ tăng tốc. Cụ thể, RDATA đảm nhận các chức năng sau:
  - **Read Input**: Đọc và lưu trữ dữ liệu Input Activation hoặc Input Feature Map vào Input Buffer.
  - **Read Weight**: Đọc và lưu trữ dữ liệu Weight hoặc Kernel vào Weight Buffer.
  - **Read Bias/Partial-Sum**: Đọc và lưu trữ dữ liệu Bias hoặc Partial-Sum vào Bias & Partial-Sum Buffer.
  - **Load Input**: Chuyển dữ liệu từ Input Buffer sang Input Register (với Fully-Connected và Convolution Layer) hoặc sang Element-Wise Unit (với Pooling Layer) để chuẩn bị cho bước tính toán.
  - **Load Weight**: Chuyển dữ liệu từ Weight Buffer sang Weight Register phục vụ tính toán.
  - **Load Bias/Partial-Sum**: Chuyển dữ liệu từ Bias & Partial-Sum Buffer vào Bias & Partial-Sum Register của Accumulate Matrix để chuẩn bị cho quá trình tính toán.
- **Compute – COMPS**: Giai đoạn này đảm nhiệm nhiệm vụ thực hiện tính toán MAC (Multiply-Accumulate) đặc trưng cho hai loại Layer: Fully-Connected và Convolution. Cụ thể gồm:
  - **Multiply-Accumulate Computation**: Thực hiện phép nhân và cộng tích lũy từ các giá trị đã được nạp vào Weight Register và Input Register bên trong Processing Matrix.
  - **Accumulate Matrix Computation**: Tổng hợp các Partial-Sum từ kết quả tính toán của Processing Matrix, sau đó cộng dồn với Bias hoặc Partial-Sum tương ứng.
- **Pooling – POOL**: Giai đoạn này chịu trách nhiệm hậu xử lý kết quả tính toán trước khi ghi ra bộ nhớ ngoài. Cụ thể gồm:
  - **Quantize**: Thực hiện lượng tử hóa kết quả đầu ra từ Accumulate Matrix, đưa dữ liệu 32-bit về dạng 8-bit.
  - **Activation**: Áp dụng các hàm kích hoạt phi tuyến (ví dụ: ReLU, ReLU6).
  - **Compare (Only in Mixed-Layer)**: Khi hệ thống chạy ở chế độ Mixed-Layer, các giá trị sau khi qua Activation sẽ được đưa vào khối Max Pooling. Tại đây, phép so sánh được thực hiện trong từng cửa sổ pooling để chọn giá trị cực đại, giúp giảm kích thước không gian của *Feature Map*.
- **Write Back – WBACK**: Giai đoạn này chịu trách nhiệm ghi kết quả tính toán cuối cùng trở lại bộ nhớ ngoài sau khi quá trình xử lý đã hoàn tất. Cụ thể, WBACK bao gồm các bước sau:
  - **Load Output Data**: Sau khi các phép tính ở giai đoạn COMPUTE (với Fully-Layer hoặc Convol-Layer) hoặc quá trình POOLING (với Pair-Layer) kết thúc, Output Buffer sẽ tải toàn bộ dữ liệu kết quả vào bộ nhớ đệm tạm, sẵn sàng cho việc ghi ra bộ nhớ ngoài.
  - **Write Memory**: Khi dữ liệu đầu ra đã sẵn sàng, Output Buffer tiến hành ghi tuần tự kết quả ra bộ nhớ ngoài, với mỗi chu kỳ ghi được 32-bit dữ liệu. Sau khi quá trình ghi hoàn tất, giai đoạn sẽ trở lại trạng thái ban đầu để chuẩn bị xử lý kết quả mới.

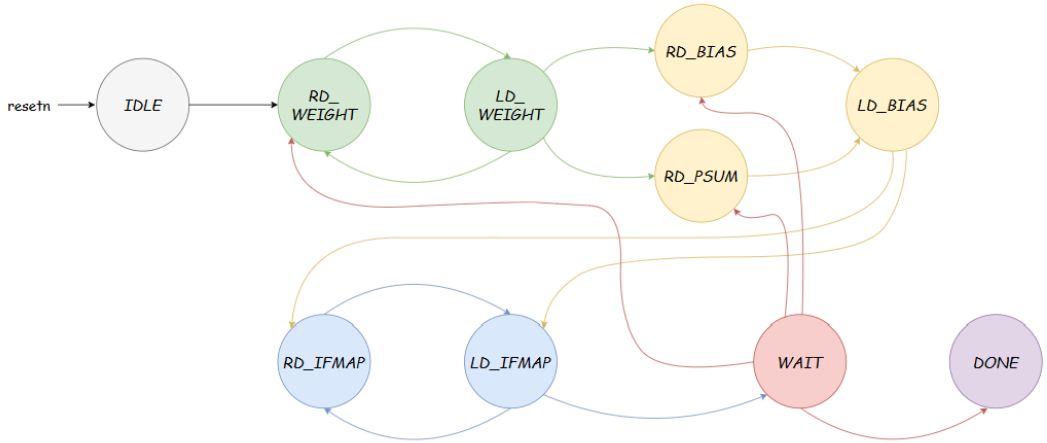
1.3.3 Máy trạng thái

Tương ứng với pipeline tối đa gồm 4 giai đoạn, bộ tăng tốc cũng được trang bị 4 máy trạng thái do khối Controller quản lý. Tùy thuộc vào chế độ hoạt động (mode) hiện tại, các máy trạng thái này sẽ được thiết kế để tối ưu theo phương pháp xử lý ưu tiên của từng mode.

1.3.3.1 Khối điều khiển RDATA

Đây là khối chịu trách nhiệm quản lý quá trình đọc và nạp dữ liệu vào hệ thống, hoạt động trên cả ba chế độ Convolution Layer, Mixed-Layer và Fully-Connected Layer.

A Convolution Layer & Mixed Layer



Hình 1.10: *Máy trạng thái ở giai đoạn RDATA của Convolution Layer*

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
IDLE	RD_WEIGHT	Chuyển sang trạng thái tiếp theo sau khi hoàn tất quá trình cấu hình ban đầu của bộ tăng tốc và nhận được tín hiệu cho biết hệ thống đã sẵn sàng hoạt động.	*Trạng thái IDLE* là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.

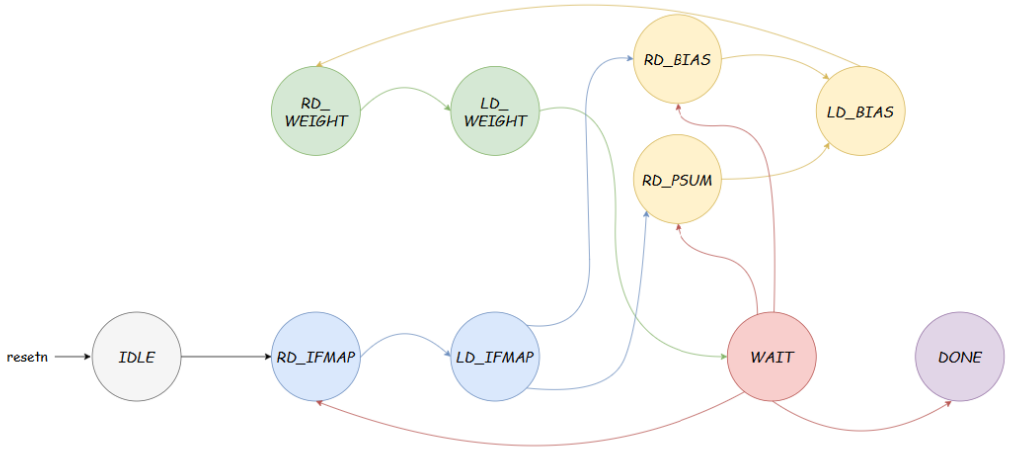
Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
RD_WEIGHT	LD_WEIGHT	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc trọng số đã hoàn tất (bao gồm cả các giá trị lượng tử hóa và tham số M0), hoặc khi lượng dữ liệu trọng số đọc vào vượt quá giới hạn tối đa mà mô hình, số thanh ghi đệm, hay bộ nhớ đệm phần cứng cho phép.	*Trạng thái RD_WEIGHT* được sử dụng để lần lượt đọc dữ liệu trọng số từ bộ nhớ ngoài vào hệ thống. Các giá trị trọng số sau khi đọc sẽ được lưu trữ trong weight buffer cục bộ.
LD_WEIGHT	RD_WEIGHT	Hoàn tất quá trình tải trọng số từ các bộ nhớ đệm vào thanh ghi, tuy nhiên số lượng trọng số hiện có vẫn chưa đủ để thực hiện phép tính, nên hệ thống quay lại trạng thái RD_WEIGHT để tiếp tục đọc.	*Trạng thái LD_WEIGHT* tải dữ liệu trọng số từ bộ nhớ đệm vào thanh ghi để chuẩn bị cho các bước tính toán tiếp theo.
	RD_BIAS	Hoàn tất quá trình tải toàn bộ giá trị trọng số từ các bộ nhớ đệm vào thanh ghi, và tập giá trị tiếp theo cần cho quá trình tính toán là BIAS, nên hệ thống chuyển sang trạng thái RD_BIAS.	
	RD_PSUM	Hoàn tất quá trình tải toàn bộ trọng số từ các bộ nhớ đệm vào thanh ghi, và tập dữ liệu tiếp theo cần thiết cho quá trình tính toán là PARTIAL-SUM, nên hệ thống chuyển sang trạng thái RD_PSUM.	
RD_BIAS	LD_BIAS	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc dữ liệu bias đã hoàn tất (bao gồm cả các giá trị lượng tử hóa và tham số M0), hoặc khi lượng dữ liệu bias đọc vào vượt quá giới hạn tối đa mà mô hình, số thanh ghi đệm hay bộ nhớ đệm phần cứng cho phép.	Trạng thái RD_BIAS được sử dụng để lần lượt đọc dữ liệu bias từ bộ nhớ ngoài vào hệ thống. Các giá trị bias sau khi đọc sẽ được lưu trữ trong bias buffer cục bộ.

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
RD_PSUM	LD_BIAS	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc dữ liệu partial-sum đã hoàn tất, hoặc khi lượng dữ liệu đọc vào vượt quá giới hạn tối đa mà mô hình, thanh ghi đệm hay bộ nhớ đệm phần cứng cho phép.	Trạng thái RD_PSUM được sử dụng để lần lượt đọc dữ liệu partial-sum từ bộ nhớ ngoài vào hệ thống. Sau khi đọc, các giá trị này sẽ được lưu trữ trong partial-sum buffer cục bộ, chuẩn bị cho giai đoạn xử lý bias tiếp theo.
LD_BIAS	RD_IFMAP	Nhận được tín hiệu cho biết quá trình tải dữ liệu bias từ khối Processing Unit đã hoàn tất và toàn bộ giá trị bias cần thiết cho chế độ hoạt động hiện tại đã được nạp đầy đủ.	Trạng thái LD_BIAS được sử dụng để truyền toàn bộ dữ liệu bias từ bias buffer vào các thanh ghi tạm của Processing Unit.
	LD_IFMAP	Hoàn tất quá trình tải các giá trị đối số hoặc partial-sum từ toàn bộ bộ nhớ đệm vào thanh ghi, và dữ liệu Input Feature Map (IFMAP) hiện tại đã đầy đủ để thực hiện lượt tính toán tiếp theo, nên hệ thống chuyển sang trạng thái LD_IFMAP.	
RD_IFMAP	LD_IFMAP	Hoàn tất quá trình đọc toàn bộ dữ liệu đầu vào (IFMAP) từ bộ nhớ ngoài và nạp chúng vào các bộ nhớ đệm của hệ thống.	Trạng thái RD_IFMAP có nhiệm vụ đọc toàn bộ dữ liệu đầu vào (IFMAP) từ bộ nhớ ngoài và nạp vào Input Buffer để chuẩn bị cho quá trình tính toán tiếp theo.
LD_IFMAP	RD_IFMAP	Hoàn tất quá trình truyền dữ liệu đầu vào từ các bộ nhớ đệm vào thanh ghi, tuy nhiên số lượng giá trị hiện có vẫn chưa đủ để thực hiện phép tính, nên hệ thống quay lại trạng thái RD_IFMAP để tiếp tục đọc thêm dữ liệu.	Trạng thái LD_IFMAP xử lý đầu vào trong giai đoạn RDATA. Nhiệm vụ của trạng thái này là truyền toàn bộ dữ liệu đầu vào từ Input Buffer vào các thanh ghi đầu vào (Input Register) để chuẩn bị cho quá trình tính toán.
	WAIT	Hoàn tất quá trình truyền toàn bộ dữ liệu đầu vào từ các bộ nhớ đệm vào thanh ghi, và số lượng giá trị hiện có đã đủ để thực hiện phép tính, nên hệ thống chuyển sang trạng thái WAIT để chờ lệnh kích hoạt xử lý tiếp theo.	

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
WAIT	RD_BIAS	Tất cả các khối điều khiển đã chuyển sang trạng thái WAIT, tuy nhiên quá trình tính toán vẫn chưa hoàn tất trên toàn bộ dữ liệu đầu vào cần thiết, và chu kỳ xử lý tiếp theo yêu cầu tập dữ liệu đối số BIAS, nên hệ thống chuyển sang trạng thái RD_BIAS để tiếp tục đọc dữ liệu này.	Trạng thái WAIT đóng vai trò đồng bộ giữa các giai đoạn trong pipeline. Trong trạng thái này, hệ thống xác nhận rằng chu kỳ xử lý của giai đoạn RDATA đã hoàn tất, và bộ điều khiển chỉ cần chờ tín hiệu hoàn tất từ các khối điều khiển còn lại trước khi chuyển sang trạng thái khác.
	RD_PSUM	Tất cả các khối điều khiển đã ở trạng thái WAIT, tuy nhiên quá trình tính toán chưa hoàn tất trên toàn bộ dữ liệu đầu vào cần thiết, và chu kỳ xử lý tiếp theo yêu cầu tập dữ liệu PARTIAL-SUM, nên hệ thống chuyển sang trạng thái RD_PSUM để tiếp tục đọc dữ liệu này.	
	RD_WEIGHT	Tất cả các khối điều khiển đã chuyển sang trạng thái WAIT và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào cần thiết, hệ thống sẵn sàng bắt đầu chu kỳ xử lý mới bằng cách quay lại trạng thái RD_WEIGHT để đọc dữ liệu trọng số cho lượt tính tiếp theo.	
	DONE	Tất cả các khối điều khiển đã ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào với tập trọng số cần thiết của layer hiện tại, nên hệ thống chuyển sang trạng thái DONE để kết thúc phiên xử lý.	
DONE			DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

Bảng 1.1: *Bảng mô tả trạng thái hoạt động ở giai đoạn RDTA của Convolution & Mixed Layer*

B Fully-Connected Layer



Hình 1.11: *Máy trạng thái ở giai đoạn RDATA của Fully-Connected Layer*

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
IDLE	RD_IFMAP	Tự động chuyển sang trạng thái RD_IFMAP ngay sau khi khởi tạo, không yêu cầu bất kỳ điều kiện kích hoạt đặc biệt nào.	Trạng thái IDLE là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.
RD_IFMAP	LD_IFMAP	Quá trình lấy dữ liệu đầu vào từ bộ nhớ ngoài đã hoàn tất và tất cả giá trị cần thiết đã được nạp vào các bộ nhớ đệm.	Trạng thái RD_IFMAP là trạng thái xử lý đầu vào của giai đoạn RDATA, với nhiệm vụ chính là đọc toàn bộ dữ liệu đầu vào từ bộ nhớ ngoài và nạp chúng vào Input Buffer để chuẩn bị cho quá trình tính toán.

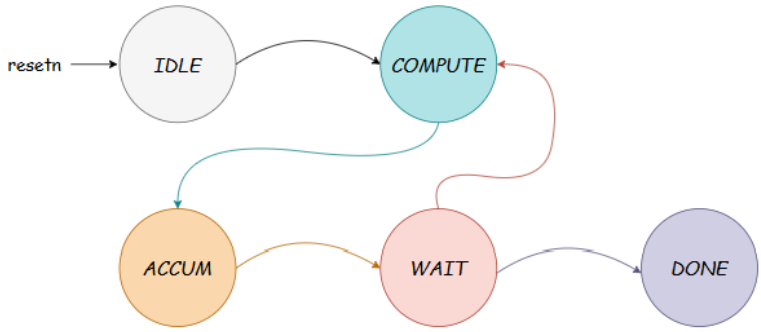
Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
LD_IFMAP	RD_BIAS	Hoàn thành quá trình tải toàn bộ giá trị đầu vào từ các bộ nhớ đệm vào thanh ghi, trong đó tập giá trị cần thiết cho quá trình tính toán là tập hệ số BIAS.	Trạng thái LD_IFMAP xử lý đầu vào trong giai đoạn RDATA. Nhiệm vụ của trạng thái này là truyền toàn bộ dữ liệu đầu vào từ Input Buffer vào các thanh ghi đầu vào (Input Register) để chuẩn bị cho quá trình tính toán.
	RD_PSUM	Hoàn tất quá trình truyền toàn bộ giá trị đầu vào từ các bộ nhớ đệm vào thanh ghi, trong đó tập giá trị phục vụ cho phép tính tiếp theo là tập tổng riêng phần (PARTIAL-SUM).	
RD_BIAS	LD_BIAS	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc dữ liệu bias đã hoàn tất (bao gồm cả các giá trị lượng tử hóa và tham số M0)	Trạng thái RD_BIAS được sử dụng để lần lượt đọc dữ liệu bias từ bộ nhớ ngoài vào hệ thống. Các giá trị bias sau khi đọc sẽ được lưu trữ trong bias buffer cục bộ.
RD_PSUM	LD_BIAS	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc dữ liệu partial-sum đã hoàn tất, hoặc khi lượng dữ liệu đọc vào vượt quá giới hạn tối đa mà mô hình, thanh ghi đệm hay bộ nhớ đệm phần cứng cho phép.	Trạng thái RD_PSUM được sử dụng để lần lượt đọc dữ liệu partial-sum từ bộ nhớ ngoài vào hệ thống. Sau khi đọc, các giá trị này sẽ được lưu trữ trong partial-sum buffer cục bộ, chuẩn bị cho giai đoạn xử lý bias tiếp theo.
LD_BIAS	RD_WEIGHT	Hoàn tất quá trình nạp các giá trị đối số hoặc tổng riêng phần từ toàn bộ bộ nhớ đệm vào các thanh ghi để chuẩn bị cho bước xử lý trọng số tiếp theo.	Trạng thái LD_BIAS được sử dụng để truyền toàn bộ dữ liệu bias từ bias buffer vào các thanh ghi tạm của Processing Unit.
RD_WEIGHT	LD_WEIGHT	Khi bộ tăng tốc nhận được tín hiệu cho biết quá trình đọc trọng số đã hoàn tất (bao gồm cả các giá trị lượng tử hóa và tham số M0)	Trạng thái RD_WEIGHT được sử dụng để lần lượt đọc dữ liệu trọng số từ bộ nhớ ngoài vào hệ thống. Các giá trị trọng số sau khi đọc sẽ được lưu trữ trong weight buffer cục bộ.
LD_WEIGHT	WAIT	Hoàn tất quá trình tải toàn bộ giá trị trọng số từ các bộ nhớ đệm vào thanh ghi	Trạng thái LD_WEIGHT được sử dụng để tải dữ liệu trọng số từ bộ nhớ đệm vào thanh ghi để chuẩn bị cho các bước tính toán tiếp theo.

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	**Điều kiện**	
WAIT	RD_BIAS	Tất cả các khối điều khiển đã chuyển sang trạng thái WAIT, tuy nhiên quá trình tính toán vẫn chưa hoàn tất trên toàn bộ dữ liệu đầu vào cần thiết, và chu kỳ xử lý tiếp theo yêu cầu tập dữ liệu đối số BIAS, nên hệ thống chuyển sang trạng thái RD_BIAS để tiếp tục đọc dữ liệu này.	Trạng thái WAIT đóng vai trò đồng bộ giữa các giai đoạn trong pipeline. Trong trạng thái này, hệ thống xác nhận rằng chu kỳ xử lý của giai đoạn RDATA đã hoàn tất, và bộ điều khiển chỉ cần chờ tín hiệu hoàn tất từ các khối điều khiển còn lại trước khi chuyển sang trạng thái khác.
	RD_PSUM	Tất cả các khối điều khiển đã ở trạng thái WAIT, tuy nhiên quá trình tính toán chưa hoàn tất trên toàn bộ dữ liệu đầu vào cần thiết, và chu kỳ xử lý tiếp theo yêu cầu tập dữ liệu PARTIAL-SUM, nên hệ thống chuyển sang trạng thái RD_PSUM để tiếp tục đọc dữ liệu này.	
	RD_IFMAP	Tất cả các khối điều khiển đều đang ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ giá trị trọng số cần thiết, sẵn sàng chuyển sang giai đoạn đọc dữ liệu đầu vào.	
	DONE	Tất cả các khối điều khiển đã ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào với tập trọng số cần thiết của layer hiện tại, nên hệ thống chuyển sang trạng thái DONE để kết thúc phiên xử lý.	
DONE			Trạng thái DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, điều đó đồng nghĩa rằng mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

Bảng 1.2: *Bảng mô tả trạng thái hoạt động ở giai đoạn RDATA của Fully-Connected Layer*

1.3.3.2 Khối điều khiển COMPS

Khối điều khiển này được thiết kế đồng nhất để thực hiện các tác vụ MAC trên cả ba chế độ: Convolution Layer, Mixed-Layer và Fully-Connected Layer.



Hình 1.12: *Máy trạng thái ở giai đoạn COMPS*

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
IDLE	COMPUTE	Khối điều khiển RDATA đã hoàn tất một chu kỳ hoạt động và hiện tại đang ở trạng thái chờ (WAIT) để chuyển sang giai đoạn tính toán tiếp theo.	Trạng thái IDLE là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.
COMPUTE	ACCUM	Processing Unit phát tín hiệu cho biết toàn bộ phép tính MAC đã hoàn tất, cho phép hệ thống chuyển sang giai đoạn cộng dồn kết quả (ACCUM).	Trạng thái COMPUTE là giai đoạn chuyên biệt đảm nhận các phép nhân và cộng tích lũy (MAC). Nhiệm vụ của trạng thái này là kết hợp toàn bộ dữ liệu đầu vào, trọng số, giá trị bias hoặc tổng riêng phần từ các giai đoạn trước để thực hiện tính toán và tạo ra kết quả tổng hợp cuối cùng phục vụ cho bước xử lý tiếp theo.

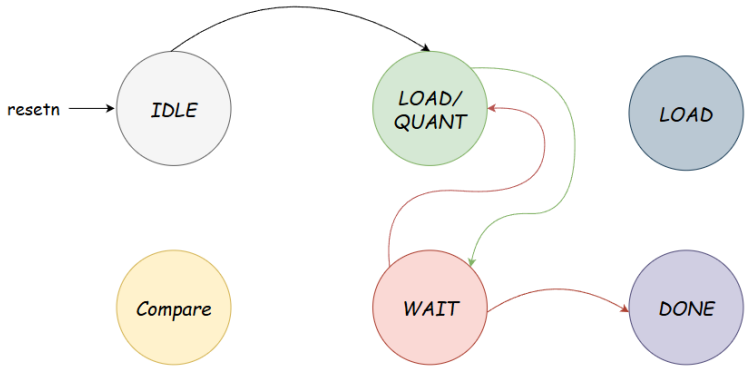
Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
ACCUM	WAIT	Hoàn tất quá trình cộng tích lũy trong khối Accumulate và hệ thống sẵn sàng chuyển sang trạng thái chờ đồng bộ (WAIT) cho các giai đoạn kế tiếp.	Trạng thái ACCUM là trạng thái chuyên biệt thuộc giai đoạn COMPS, chịu trách nhiệm thực hiện phép cộng tích lũy các tổng riêng phần. Nhiệm vụ của trạng thái này là thu nhận toàn bộ kết quả trung gian từ ProcessingMatrix và cộng dồn các tổng riêng phần tương ứng để hình thành tổng tích lũy lớn hơn hoặc tổng kết quả cuối cùng cho đầu ra.
WAIT	COMPUTE	Tất cả các khối điều khiển đều đang ở trạng thái WAIT, quá trình tính toán vẫn chưa hoàn tất trên toàn bộ giá trị đầu vào, trọng số và tổng riêng phần hoặc đối số cần thiết, và các giá trị này đã được lưu trữ sẵn trong các thanh ghi để chuẩn bị cho giai đoạn tính toán tiếp theo.	Trạng thái WAIT đóng vai trò đồng bộ giữa các giai đoạn trong pipeline. Trong trạng thái này, hệ thống xác nhận rằng chu kỳ xử lý của giai đoạn COMPS đã hoàn tất, và bộ điều khiển chỉ cần chờ tín hiệu hoàn tất từ các khối điều khiển còn lại trước khi chuyển sang trạng thái khác.
	DONE	Tất cả các khối điều khiển đã ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào với tập trọng số cần thiết của layer hiện tại, nên hệ thống chuyển sang trạng thái DONE để kết thúc phiên xử lý.	
DONE			Trạng thái DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, điều đó đồng nghĩa rằng mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

Bảng 1.3: *Bảng mô tả trạng thái hoạt động ở giai đoạn COMPS*

1.3.3.3 Khối điều khiển POOL

Khối điều khiển này được thiết kế cho việc hậu xử lý kết quả tính toán cho cả ba layer Convolution Layer, Fully-Connected Layer và Mixed Layer

A Convolution Layer & Fully-Connected Layer



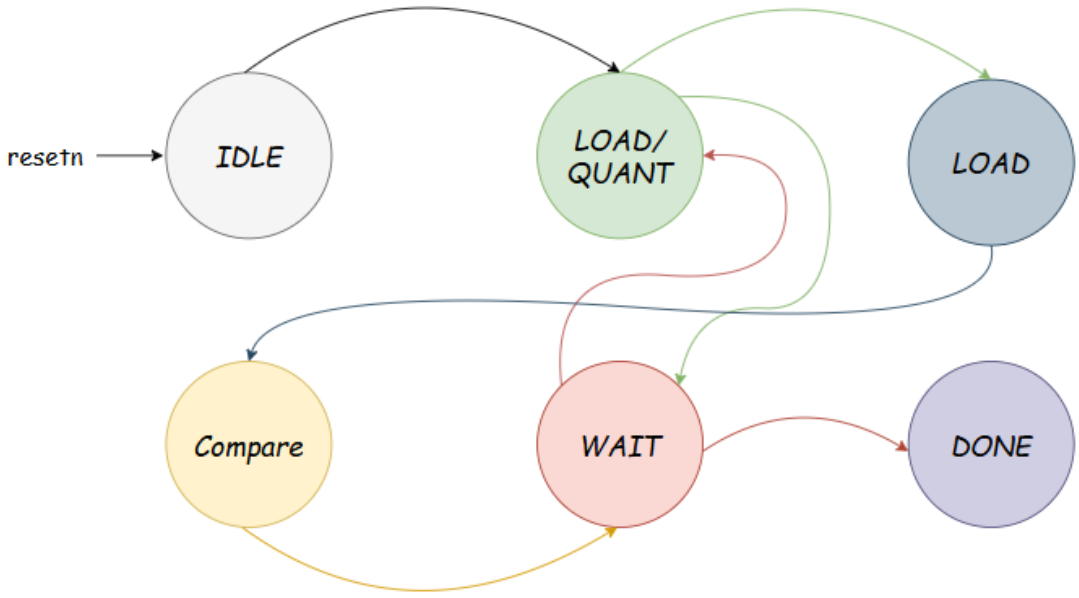
Hình 1.13: *Máy trạng thái ở giai đoạn POOL của Convolution và Fully-Connected Layer*

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
IDLE	LOAD/QUANT	Khối điều khiển RDATA đã hoàn tất hai chu kỳ hoạt động và khối điều khiển COMPS dã hoàn tất một chu kỳ hoạt đọng và hiện tại cả hai đang ở trạng thái chờ (WAIT) để chuyển sang giai đoạn tính toán tiếp theo.	Trạng thái IDLE là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.
LOAD/QUANT	WAIT	Khi hành vi của trạng thái là LOAD - Hoàn thành việc tải giá trị kết quả đầu ra vào bộ nhớ đệm để chuẩn bị cho quá trình WBACK. Khi hành vi của trạng thái là QUANT - Hoàn thành việc lượng tử hóa kết quả đầu ra.	*Trạng thái QUANT/LOAD* là trạng thái đặc biệt, quyết định thực hiện *LOAD* dữ liệu ra WBACK hay *Quantize* đầu ra. Nếu kết quả là tổng riêng phần thì thực hiện *LOAD*, còn nếu kết quả là tổng cuối cùng thì thực hiện *Quantize*.

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
WAIT	LOAD/QUANT	Tất cả các khối điều khiển đều đang ở trạng thái WAIT, quá trình tính toán vẫn chưa hoàn tất trên toàn bộ giá trị đầu vào.	Trạng thái WAIT đóng vai trò đồng bộ giữa các giai đoạn trong pipeline. Trong trạng thái này, hệ thống xác nhận rằng chu kỳ xử lý của giai đoạn POOL đã hoàn tất, và bộ điều khiển chỉ cần chờ tín hiệu hoàn tất từ các khối điều khiển còn lại trước khi chuyển sang trạng thái khác.
	DONE	Tất cả các khối điều khiển đã ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào với tập trọng số cần thiết của layer hiện tại, nên hệ thống chuyển sang trạng thái DONE để kết thúc phiên xử lý.	
DONE			Trạng thái DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, điều đó đồng nghĩa rằng mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

Bảng 1.4: *Bảng mô tả trạng thái hoạt động ở giai đoạn POOL của Convolution và Fully-Connected Layer*

B Mixed Layer



Hình 1.14: *Máy trạng thái ở giai đoạn POOL của Mixed Layer*

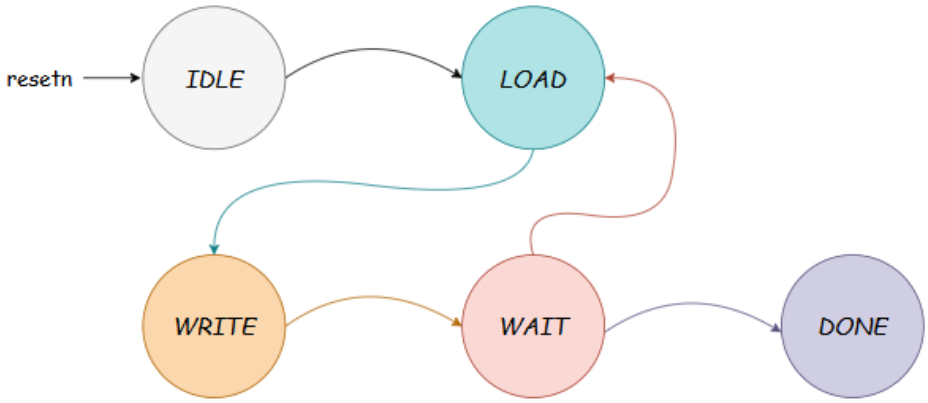
Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	**Điều kiện**	
IDLE	LOAD/QUANT	Khối điều khiển RDATA đã hoàn tất hai chu kỳ hoạt động và khối điều khiển COMPS dã hoàn tất một chu kỳ hoạt đọng và hiện tại cả hai đang ở trạng thái chờ (WAIT) để chuyển sang giai đoạn tính toán tiếp theo.	Trạng thái IDLE là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.
LOAD/QUANT	LOAD	Khi hệ thống hiện tại tính toán tổng cuối cùng và hành vi của trạng thái là QUANT đã hoàn thành việc lượng tử hóa.	*Trạng thái LOAD/QUANT* là trạng thái đặc biệt, quyết định việc *LOAD* dữ liệu ra WBACK hay thực hiện *Quantize*. Nếu kết quả là tổng riêng phần thì thực hiện *LOAD*, còn nếu kết quả là tổng cuối cùng thì thực hiện *Quantize*.
	WAIT	Khi hệ thống tính toán ra tổng riêng phần và hành vi của trạng thái là LOAD đã hoàn thành	
LOAD	COMPARE	Hoàn thành quá trình tải giá trị kết quả vào bộ nhớ đệm của khối Max Pooling	*Trạng thái LOAD* là trạng thái đưa dữ liệu đầu ra sau lượng tử hóa vào buffer để khối Max Pooling thực hiện so sánh.
COMPARE	WAIT	Khi quá trình so sánh trong khối Max Pooling đã hoàn tất và tìm được giá trị lớn nhất	*Trạng thái COMPARE* thực hiện so sánh các giá trị trong buffer của khối Max Pooling để xác định giá trị cực đại.
WAIT	LOAD/QUANT	Tất cả các khối điều khiển đều đang ở trạng thái WAIT, quá trình tính toán vẫn chưa hoàn tất trên toàn bộ giá trị đầu vào.	Trạng thái WAIT đóng vai trò đồng bộ giữa các giai đoạn trong pipeline. Trong trạng thái này, hệ thống xác nhận rằng chu kỳ xử lý của giai đoạn POOL đã hoàn tất, và bộ điều khiển chỉ cần chờ tín hiệu hoàn tất từ các khối điều khiển còn lại trước khi chuyển sang trạng thái khác.
	DONE	Tất cả các khối điều khiển đã ở trạng thái WAIT, và quá trình tính toán đã hoàn tất trên toàn bộ dữ liệu đầu vào với tập trọng số cần thiết của layer hiện tại, nên hệ thống chuyển sang trạng thái DONE để kết thúc phiên xử lý.	

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
DONE			Trạng thái DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, điều đó đồng nghĩa rằng mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

Bảng 1.5: *Bảng mô tả trạng thái hoạt động ở giai đoạn POOl của Mixed Layer*

1.3.3.4 Khối điều khiển WBACK

Khối điều khiển này được thiết kế để thực hiện việc đọc dữ liệu trên cả ba chế độ: Convolution Layer, Mixed-Layer và Fully-Connected Layer.



Hình 1.15: *Máy trạng thái ở giai đoạn WBACK của Convolution Layer*

Trạng thái hiện tại	Chuyển đổi trạng thái		Mô tả hành vi
	Trạng thái tiếp theo	Điều kiện	
IDLE	LOAD	Khối điều khiển RDATA đã hoàn tất một chu kỳ hoạt động và hiện tại đang ở trạng thái WAIT. (Lưu ý: Pooling Layer chỉ thực hiện hoạt động pipeline trong 2 giai đoạn.)	Trạng thái IDLE là trạng thái khởi đầu của máy trạng thái và cũng là trạng thái mà hệ thống quay lại sau khi reset. Nhiệm vụ chính của IDLE là duy trì hệ thống ở trạng thái sẵn sàng, đảm bảo tất cả thanh ghi và tín hiệu điều khiển được khởi tạo ổn định với giá trị mặc định, chuẩn bị cho một chu kỳ xử lý mới khi có tín hiệu kích hoạt.
LOAD	WRITE	Hoàn tất quá trình chuyển các giá trị đầu ra vào toàn bộ các bộ nhớ đệm để sẵn sàng cho giai đoạn ghi kết quả.	Trạng thái COMPUTE là giai đoạn chuyên biệt đảm nhận các phép nhân và cộng tích lũy (MAC). Nhiệm vụ của trạng thái này là kết hợp toàn bộ dữ liệu đầu vào, trọng số, giá trị bias hoặc tổng riêng phần từ các giai đoạn trước để thực hiện tính toán và tạo ra kết quả tổng hợp cuối cùng phục vụ cho bước xử lý tiếp theo.
WRITE	WAIT	Hoàn tất quá trình truyền dữ liệu từ các bộ nhớ đệm nội bộ ra bộ nhớ ngoài, hoàn thành bước ghi kết quả.	Trạng thái ACCUM là trạng thái chuyên biệt thuộc giai đoạn COMPS, chịu trách nhiệm thực hiện phép cộng tích lũy các tổng riêng phần. Nhiệm vụ của trạng thái này là thu nhận toàn bộ kết quả trung gian từ Processing Matrix và cộng dồn các tổng riêng phần tương ứng để hình thành tổng tích lũy lớn hơn hoặc tổng kết quả cuối cùng cho đầu ra.

Trạng thái hiện tại	**Chuyển đổi trạng thái**		**Mô tả hành vi**
	Trạng thái tiếp theo	**Điều kiện**	
WAIT	LOAD	Tất cả các khối điều khiển đang ở trạng thái WAIT, và quá trình ghi dữ liệu vẫn chưa hoàn tất trên toàn bộ giá trị đầu ra của lớp, do đó hệ thống chuyển sang chu kỳ tải dữ liệu kết quả vào bộ nhớ đệm.	Trạng thái LD_IFMAP xử lý đầu vào trong giai đoạn RDATA. Nhiệm vụ của trạng thái này là truyền toàn bộ dữ liệu đầu vào từ Input Buffer vào các thanh ghi đầu vào (Input Register) để chuẩn bị cho quá trình tính toán.
	DONE	Tất cả các khối điều khiển đang ở trạng thái WAIT, và quá trình ghi toàn bộ dữ liệu đầu ra của lớp đã hoàn tất, do đó hệ thống kết thúc chu kỳ hoạt động và chuyển sang trạng thái DONE.	
DONE			Trạng thái DONE là trạng thái kết thúc của toàn bộ máy trạng thái. Khi đạt đến trạng thái này, điều đó đồng nghĩa rằng mọi tác vụ liên quan đến quá trình đọc dữ liệu trong giai đoạn WBACK đã được hoàn tất hoàn toàn.

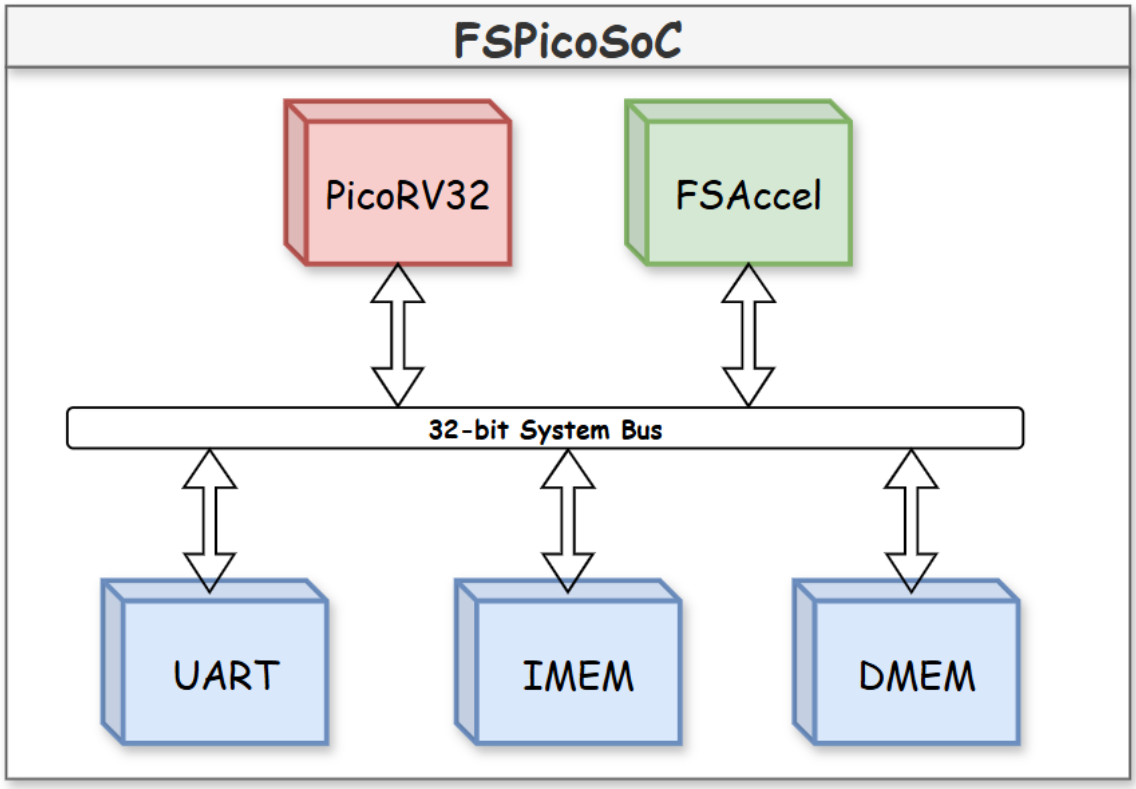
Bảng 1.6: *Bảng mô tả trạng thái hoạt động ở giai đoạn WBACK*

Chương 2

Thiết kế hệ thống

2.1 Kiến trúc phần cứng

Hệ thống FSPicoSoC được phát triển dựa trên nền tảng PicoSoC, vốn sử dụng lõi xử lý RISC-V PicoRV32 với kiến trúc đơn giản, dễ tinh chỉnh. Trên cơ sở đó, hệ thống được tích hợp thêm bộ tăng tốc Flash Accelerator, giúp đẩy nhanh các tác vụ tính toán song song. Nhờ vậy, FSPicoSoC vừa duy trì sự gọn nhẹ của PicoSoC, vừa đạt hiệu năng cao hơn nhờ phần cứng tăng tốc chuyên dụng. Hình 2.1 minh họa kiến trúc tổng thể của hệ thống.



Hình 2.1: *Sơ đồ khối về kiến trúc tổng quát của hệ thống FSPicoSoC*

FSPicoSoC bao gồm các thành phần chính:

- **PicoRV32**: Lõi xử lý trung tâm (CPU), chịu trách nhiệm điều khiển và thực thi mã lệnh RISC-V.

- **FSAccel (Flash Accelerator)**: Bộ tăng tốc phần cứng, đảm nhiệm các tác vụ tính toán nặng (ví dụ: Convolution, Fully-Connected).
- **IMEM (Instruction Memory)**: Bộ nhớ lệnh dung lượng 256 KB, lưu trữ chương trình thực thi.
- **DMEM (Data Memory)**: Bộ nhớ dữ liệu dung lượng 128 KB, lưu trữ dữ liệu trung gian trong quá trình xử lý.
- **UART**: Ngoại vi giao tiếp nối tiếp, trong thiết kế thực tế được dùng để trao đổi dữ liệu với máy tính hoặc thiết bị ngoài. UART chủ yếu có vai trò *xuất dữ liệu ra ngoài để quan sát và xác minh hệ thống*, giúp đánh giá việc SoC thực thi chương trình có đúng như mong đợi hay không.
- **32-bit System Bus**: Bus hệ thống, kết nối và truyền dữ liệu giữa PicoRV32, FSAccel, IMEM, DMEM và UART.

2.2 Nền tảng phần mềm

Bên cạnh phần cứng, hệ thống SoC cần có một nền tảng phần mềm tương thích để hỗ trợ hoạt động và khai thác tối đa khả năng xử lý và đồng bộ để hỗ trợ hoạt động của SoC.
Đối với FSPicoSoC - hệ thống SoC được thiết kế nhằm thực hiện tác vụ suy luận trên mô hình Deep Learning - nền tảng phần mềm đi kèm bao gồm các thành phần chính sau:

- Bộ công cụ phần mềm hỗ trợ quá trình huấn luyện mô hình Deep Learning.
- Bộ công cụ phục vụ việc biên dịch mã nguồn C thành mã máy.
- Tập thư viện ngôn ngữ C hỗ trợ thực thi tác vụ suy luận mô hình Deep Learning.
- Tập thư viện ngôn ngữ C được phát triển riêng cho FSPicoSoC, đảm nhiệm các chức năng cơ bản và thay thế một số thư viện C chuẩn (ví dụ: các tác vụ nhập/xuất – I/O).

2.2.1 Bộ công cụ phần mềm hỗ trợ quá trình huấn luyện mô hình Deep Learning

Trong hệ thống này, bộ công cụ phần mềm được lựa chọn để hỗ trợ quá trình huấn luyện và triển khai mô hình Deep Learning là **TensorFlow Lite** – một nền tảng phổ biến và mạnh mẽ do Google phát triển, được tối ưu hóa đặc biệt cho các thiết bị nhúng, thiết bị biên và di động.
TensorFlow Lite cung cấp một tập công cụ toàn diện giúp người dùng triển khai mô hình học máy trực tiếp trên thiết bị với nhiều ưu điểm nổi bật như:

- **Hiệu năng và độ trễ thấp**: Mô hình được thực thi trực tiếp trên thiết bị mà không cần giao tiếp với máy chủ, từ đó giảm độ trễ và nâng cao tốc độ phản hồi.
- **Bảo mật dữ liệu**: Dữ liệu được xử lý nội bộ trên thiết bị, giúp đảm bảo tính riêng tư.
- **Tối ưu hóa kích thước và năng lượng**: Hỗ trợ giảm kích thước mô hình và nhị phân, đồng thời tối ưu việc tiêu thụ năng lượng khi suy luận.
- **Khả năng tương thích cao**: Hoạt động tốt trên nhiều nền tảng phần cứng như Android, iOS, Linux nhúng và vi xử lý; đồng thời hỗ trợ nhiều ngôn ngữ lập trình như C/C++, Python, Java và Swift.
- **Tận dụng tăng tốc phần cứng**: Cải thiện hiệu suất suy luận thông qua các kỹ thuật tối ưu hóa mô hình và hỗ trợ tăng tốc phần cứng.

Bên cạnh đó, **TensorFlow Lite** còn cung cấp nhiều ví dụ triển khai sẵn cho các tác vụ học máy thông dụng như phân loại hình ảnh, nhận diện đối tượng hay phân tích chữ viết tay, giúp quá trình phát triển và thử nghiệm mô hình trở nên dễ dàng hơn.
Lý do chính để lựa chọn **TensorFlow Lite** cho hệ thống ... là vì nó đáp ứng đầy đủ các yêu cầu của một nền tảng triển khai mô hình học máy tại thiết bị, có cộng đồng hỗ trợ rộng lớn, và đặc biệt là hỗ trợ lượng tử hóa mô hình về số nguyên 8-bit – một yếu tố then chốt để nhóm tham khảo và hiện thực trực tiếp trong phần cứng của hệ thống.

2.2.2 Bộ công cụ phục vụ việc biên dịch mã nguồn C thành mã máy

Trong trường hợp của ..., với lõi xử lý là PicoRV32 dựa trên kiến trúc RISC-V, công cụ biên dịch không thể thiếu chính là RISC-V GNU Compiler Toolchain.
RISC-V GNU Compiler Toolchain là bộ công cụ mã nguồn mở dùng để biên dịch các chương trình viết bằng C/C++ thành mã máy có thể chạy trực tiếp trên vi xử lý RISC-V. Đây là một công cụ miễn phí, linh hoạt và được hỗ trợ rộng rãi, phù hợp với mọi thiết bị sử dụng tập lệnh RISC-V ISA.

2.2.3 Tập thư viện ngôn ngữ C hỗ trợ thực thi tác vụ suy luận mô hình Deep Learning

Để thực hiện tác vụ suy luận mô hình Deep Learning trên hệ thống ..., cần xây dựng một thư viện ngôn ngữ C cung cấp các hàm và giao thức cơ bản hỗ trợ cho quá trình tính toán. Thay vì sử dụng trực tiếp các thư viện phức tạp như TensorFlow Lite hay Caffe, nhóm lựa chọn phát triển một thư viện tối giản nhằm dễ dàng kiểm soát và tùy chỉnh cho việc tích hợp với **Flash Accelerator**.
Thư viện này được xây dựng dựa trên hai nguồn chính:

- **TensorFlow Lite**: tham khảo cách hiện thực các lớp (Layer) và xử lý dữ liệu lượng tử hóa 8-bit.
- **gemmlowp**: tham khảo các phép tính tích chập ma trận độ chính xác thấp.

Hiện tại, thư viện hỗ trợ tính toán với dữ liệu *int8_t* cho ba lớp cơ bản: Convolution Layer, Fully-Connected Layer, và Max Pooling Layer, hoạt động ở cả hai chế độ có hoặc không sử dụng Flash Accelerator, với cách triển khai tối giản dựa trên nguyên lý tính toán của từng lớp.

2.2.4 Tập thư viện ngôn ngữ C được phát triển riêng cho FSPicoSoC

Bên cạnh các thư viện hỗ trợ mô hình Deep Learning và các công cụ biên dịch, hệ thống ... cũng cần một tập thư viện ngôn ngữ C được phát triển riêng để đảm bảo phần mềm có thể giao tiếp hiệu quả với phần cứng. Điều này xuất phát từ thực tế rằng nhiều thư viện chuẩn trong ngôn ngữ C, đặc biệt là các hàm liên quan đến I/O, không thể hoạt động đúng trong môi trường phần cứng tùy chỉnh của SoC.
Chính vì vậy, một tập thư viện chuyên dụng đã được xây dựng nhằm thay thế các chức năng I/O chuẩn, cho phép truy xuất và trao đổi dữ liệu với các thiết bị ngoại vi hoặc khối UART thông qua các địa chỉ nhớ được định nghĩa sẵn. Bên cạnh đó, thư viện này còn hỗ trợ cấu hình, điều khiển và tương tác với **Flash Accelerator**, giúp thực thi các tác vụ suy luận mô hình Deep Learning ở cấp độ Layer.

Chương 3

Hiện thực

3.1 Hiện thực huấn luyện mô hình

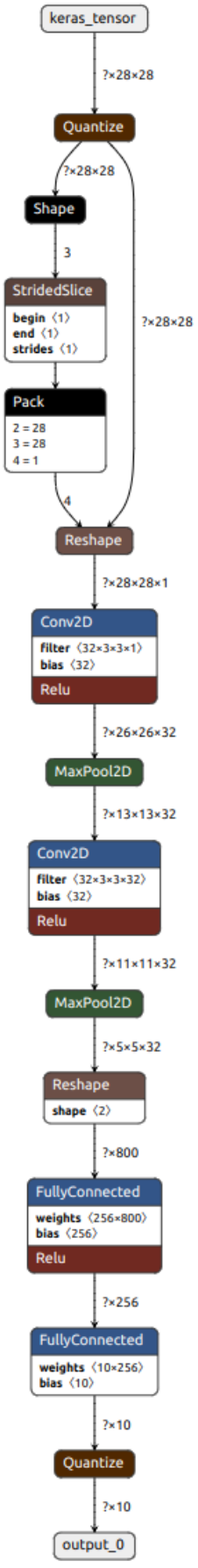
Tiến hành huấn luyện mô hình trên tập MNIST bằng TensorFlow Lite. Bên cạnh đó, cũng tự hiện thực lại một mô hình CNN cơ bản bằng Python chạy trên nền tảng Jupyter Notebook, với luồng tính toán và điều phối dữ liệu tương tự như source code C chạy trên PicoRV32 processor và code Verilog để hiện thực Accelerator. Mục tiêu là kiểm tra độ chính xác huấn luyện mô hình, theo dõi chi tiết kết quả tính toán tại từng lớp trong source code Python, đồng thời so sánh độ chính xác mô hình với TensorFlow Lite. Source code Python này sẽ là cơ sở để phát triển source code C trên PicoRV32 và code Verilog cho Flash Accelerator.
Các thành phần trong model hiện thực bằng Python gồm:

- **Dữ liệu MNIST**: chứa dữ liệu đầu vào cho các layer (input feature map, filter, bias, multiplier, scale,...)
- **2 lớp Convolution Layer**
- **2 lớp Max Pooling Layer**
- **2 lớp Fully-Connected Layer**

Các phép tính trong từng lớp, chẳng hạn lượng tử hóa, nhân tích chập ở Convolutional Layer, nhân vô hướng ở Fully Connected Layer, hay Max Pooling,... đều được hiện thực lại dựa trên các công thức đã được phân tích ở phần Cơ sở lý thuyết. Bên dưới là mô tả của model kiểm thử:

Tên	**Layer**	**Kích thước đầu ra**	**Số lượng tham số**
reshape	Reshape	$28 \times 28 \times 1$	0
conv2d	Conv2D	$26 \times 26 \times 32$	320
max_pooling2d	MaxPooling2D	$13 \times 13 \times 32$	0
conv2d_1	Conv2D	$11 \times 11 \times 32$	9284
max_pooling2d_1	MaxPooling2D	$5 \times 5 \times 32$	0
flatten	Flatten	800	0
dense	Dense	256	205506
dense_1	Dense	10	2570

Bảng 3.1: *Mô tả của model kiểm thử*



Hình 3.1: *Sơ đồ của model kiểm thử sau khi thực hiện xong quá trình lượng tử hóa*

Mô hình sau khi huấn luyện và chuyển sang TensorFlow Lite, được lượng tử hóa xuống *8-bit integer*, đạt độ chính xác như sau:

- *Mô hình gốc 32-bit floating-point*: 99.12%
- *Mô hình 8-bit integer*: 99.10%

Kết quả đánh giá trên tập kiểm thử MNIST gồm 10.000 mẫu cho thấy, mô hình lượng tử hóa chỉ giảm 0.02% độ chính xác so với mô hình gốc.

3.2 Hiện thực bộ tăng tốc Flash Accelerator

3.2.1 Kết quả mô phỏng

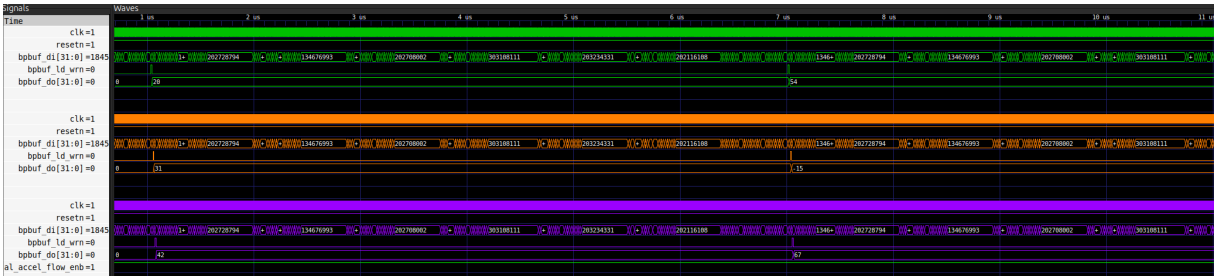
Để kiểm chứng hoạt động của Flash Accelerator, chúng tôi tiến hành mô phỏng trên ngữ cảnh Mixed Layer Mode, trong đó dữ liệu được xử lý tuần tự từ *Convolutional Layer* đến *Max Pooling Layer*. Các cấu hình mô phỏng được thiết lập như sau:

- **Kích thước dữ liệu đầu vào và trọng số:**
  - Input Feature Map: $8 \times 8 \times 3$
  - Kernel: $3 \times 3 \times 3 \times 6$
  - Output Feature Map: $3 \times 3 \times 6$
  - Bias: 6 phần tử
- **Thiết lập vùng địa chỉ bộ nhớ:**
  - Vùng nhớ Input Feature Map: địa chỉ 0000
  - Vùng nhớ Kernel: địa chỉ 6000
  - Vùng nhớ Bias: địa chỉ 5600
  - Vùng nhớ Output Feature Map: địa chỉ 16000
  - Vùng nhớ Partial-Sum: địa chỉ 20000
- **Cấu hình xử lý:**
  - Hàm kích hoạt: ReLU
  - Tham số lượng tử hoá cho từng channel:
    - Channel 1: $M_0 = 2039693188,\ n = 8$
    - Channel 2: $M_0 = 2097238482,\ n = 8$
    - Channel 3: $M_0 = 1378465373,\ n = 8$
    - Channel 4: $M_0 = 1543907582,\ n = 8$
    - Channel 5: $M_0 = 1858862255,\ n = 8$
    - Channel 6: $M_0 = 1117338165,\ n = 8$
  - Input Offset: 128
  - Output Offset: 128

Chi tiết mô phỏng đối với từng thành phần của Accelerator như sau:

1. **Thực hiện đọc và ghi giá trị vào các Buffer trong RDATA:**

- *Bias and Partial-Sum Buffer*: Trong Flash Accelerator có tổng cộng 3 khối *Bias and Partial-Sum Buffer*, được minh họa trong waveform bằng các màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối cuối cùng). Hình 3.2 thể hiện hoạt động của các buffer này trong quá trình mô phỏng:

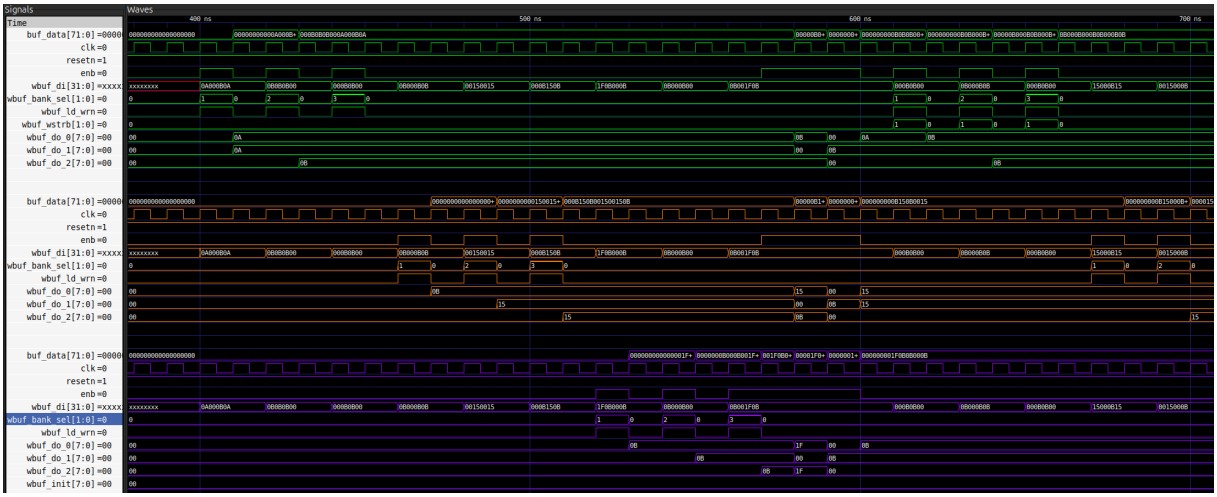


Hình 3.2: *Waveform của Bias and Partial-Sum Buffer*

Phân tích waveform của Bias and Partial-Sum Buffer:

  - Trong giai đoạn 1020 ns đến 1050 ns, tín hiệu điều khiển *bpbuf_ld_wrn* được kích hoạt, cho phép dữ liệu *bpbuf_di* được ghi vào buffer. Sau đó, từ 1050 ns đến 7010 ns, dữ liệu đầu ra *bpbuf_do* giữ nguyên cho đến khi xuất hiện một lần ghi mới.
  - Do cả giá trị Bias và Partial-Sum đều có độ rộng 32-bit, nên mỗi buffer chỉ cần ghi duy nhất một dữ liệu trong mỗi chu kỳ tính toán đối với mỗi buffer.

- *Weight Buffer*: Trong Flash Accelerator có tổng cộng 3 khối *Weight Buffer*, được phân biệt trong waveform bằng các màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba). Hình 3.4 minh họa hoạt động của các buffer này trong quá trình mô phỏng:

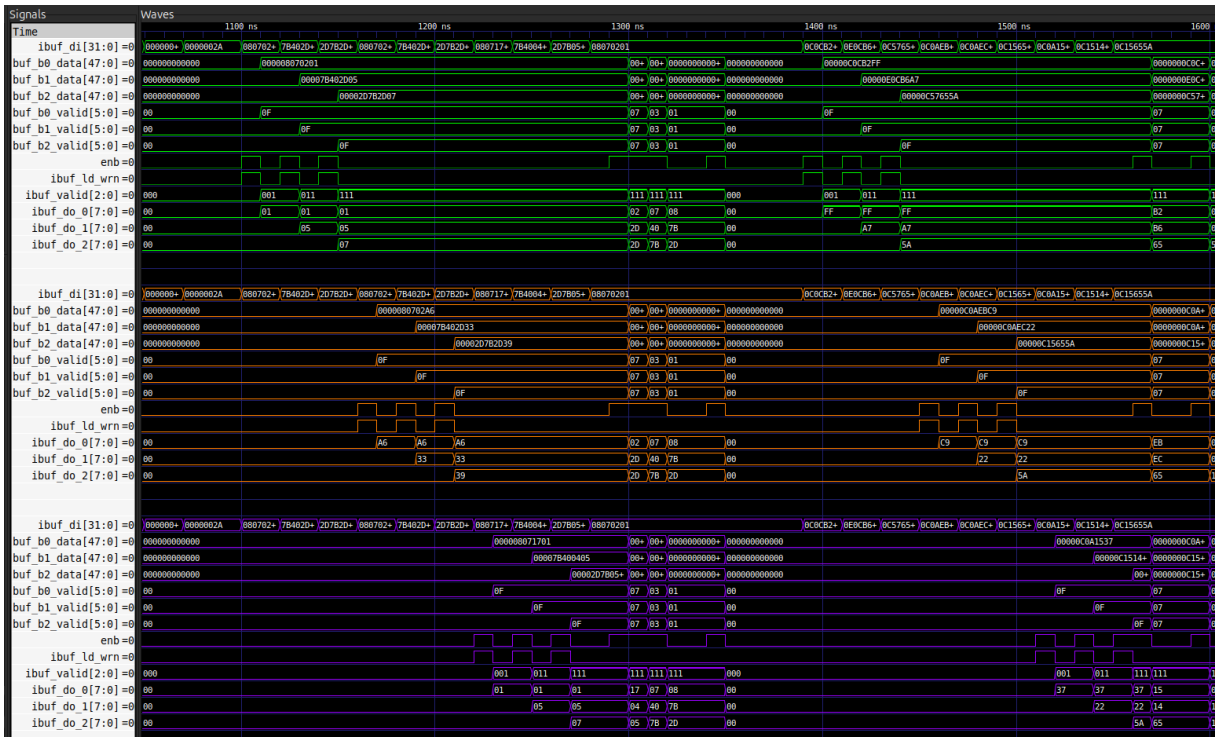


Hình 3.3: *Waveform của Weight Buffer*

Phân tích waveform của Weight Buffer:

  - Trong khoảng 400 ns đến 570 ns, quá trình đọc dữ liệu diễn ra được điều khiển bởi tín hiệu wbuf_ld_wrn. Dữ liệu được nạp tuần tự vào các buffer: đầu tiên là buffer màu xanh, tiếp theo là cam, và cuối cùng là tím. Sau khi đọc, dữ liệu được lưu trữ trong biến *buf_data* của các buffer.
  - Từ 570 ns đến 610ns, quá trình ghi dữ liệu bắt đầu. Các giá trị trong *buf_data* được xuất ra để phục vụ cho giai đoạn tính toán tại Processing Array.
  - Giai đoạn 610 ns trở về sau sẽ lặp lại quá trình đọc và ghi dữ liệu như được trình bày phía trên

- *Input Buffer*: Trong Flash Accelerator có tổng cộng 3 khối Input Buffer, được phân biệt trong waveform bằng các màu: : xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba).



Hình 3.4: *Waveform của Input Buffer*

Phân tích waveform của Input Buffer:

- Trong khoảng thời gian từ 1100 ns đến 1270 ns, tín hiệu *ibuf_ld_wrn* điều khiển quá trình đọc dữ liệu, các giá trị đầu vào được nạp vào ba bộ nhớ đệm tương ứng trong mỗi buffer.
- Trong khoảng từ 1290 ns đến 1320 ns, hệ thống thực hiện ghi dữ liệu, các giá trị được xuất ra từ bộ nhớ đệm ra ngoài.
- Từ 1320 ns đến 1600 ns, quá trình đọc–ghi dữ liệu được lặp lại theo chu kỳ.

2. Thực hiện tác vụ tính toán trong COMPS:

- *Processing Matrix*: Trong Flash Accelerator có một khối *Processing Matrix*, chịu trách nhiệm thực hiện các phép tính tích chập như đã mô tả trong phần kiến trúc. Với quy ước cấu hình $(L_w, L_h, L_c, L_k) = (3, 3, 3, 3)$, số lượng dữ liệu tham gia tính toán được xác định như sau:
  - Số lượng trọng số (Weight) cần thiết:

$$L_w \times L_h \times L_c \times L_k = 3 \times 3 \times 3 \times 3 = 81$$

  - Số lượng phần tử đầu vào (Input Feature Map):

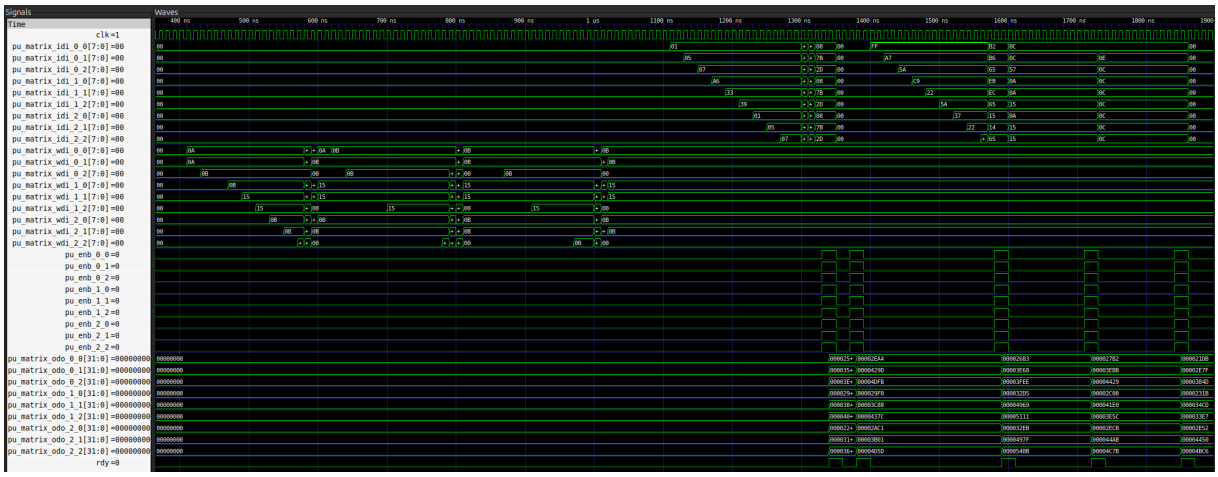
$$L_w \times L_h \times L_c = 3 \times 3 \times 3 = 27$$

  - Tổng số kết quả đầu ra thu được sau phép tích chập:

$$\frac{81}{9} = 9$$

  (tương ứng với 9 phép nhân–cộng độc lập).

Hình 3.5 minh họa waveform của khối Processing Matrix trong quá trình mô phỏng.

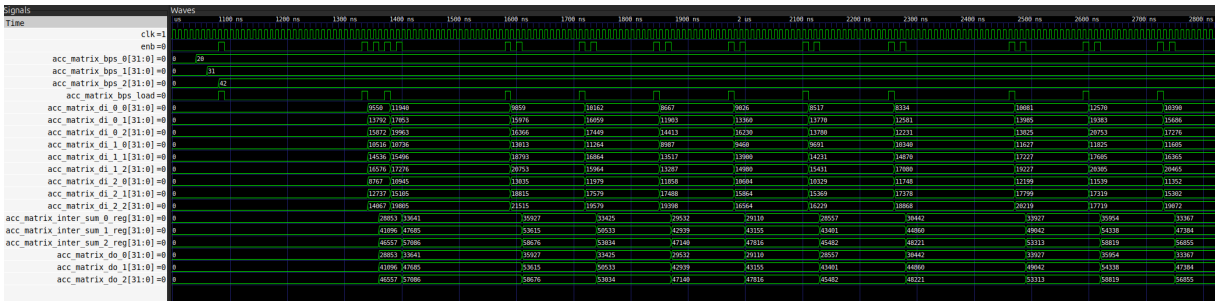


Hình 3.5: *Waveform của Processing Matrix*

Phân tích waveform của Processing Matrix:

- Trong khoảng thời gian 400 ns đến 1050 ns, các giá trị trọng số được nạp từ *Weight Buffer* và lưu trữ vào bộ đệm nội bộ của Processing Matrix.
- Từ 1110 ns đến 1330ns, dữ liệu đầu vào được lấy từ *Input Buffer* và ghi vào các thanh ghi bên trong Processing Matrix, chuẩn bị cho giai đoạn tính toán.
- Tại 1330 ns đến 1350 ns, khi cả trọng số và dữ liệu đầu vào đã được nạp đầy đủ, các *Processing Unit* (PU) bên trong Processing Matrix bắt đầu hoạt động và tạo ra những kết quả đầu tiên.
- Trong khoảng 1350 ns đến 1600ns, quá trình tính toán được tiến hành liên tục. Kernel lần lượt trượt qua các vị trí của Input Feature Map, vì vậy chỉ cần thay đổi dữ liệu giá trị đầu vào và tính toán.

- *Accumulate Matrix*: Trong Flash Accelerator có một khối Accumulate Matrix, chịu trách nhiệm cộng dồn các kết quả trung gian từ Processing Matrix. Cụ thể, 9 giá trị tổng riêng phần (partial sums) được sinh ra từ các *Processing Unit* sẽ được kết hợp với giá trị *Bias* và (nếu có) các tổng riêng phần từ chu kỳ trước đó. Kết quả cuối cùng sau giai đoạn này là dữ liệu đã được tích lũy đầy đủ, chuẩn bị cho bước hậu xử lý tại khối POOL.
  Hình 3.6 minh họa waveform của khối Accumulate Matrix trong quá trình mô phỏng.



Hình 3.6: *Waveform của Accumulate Matrix*

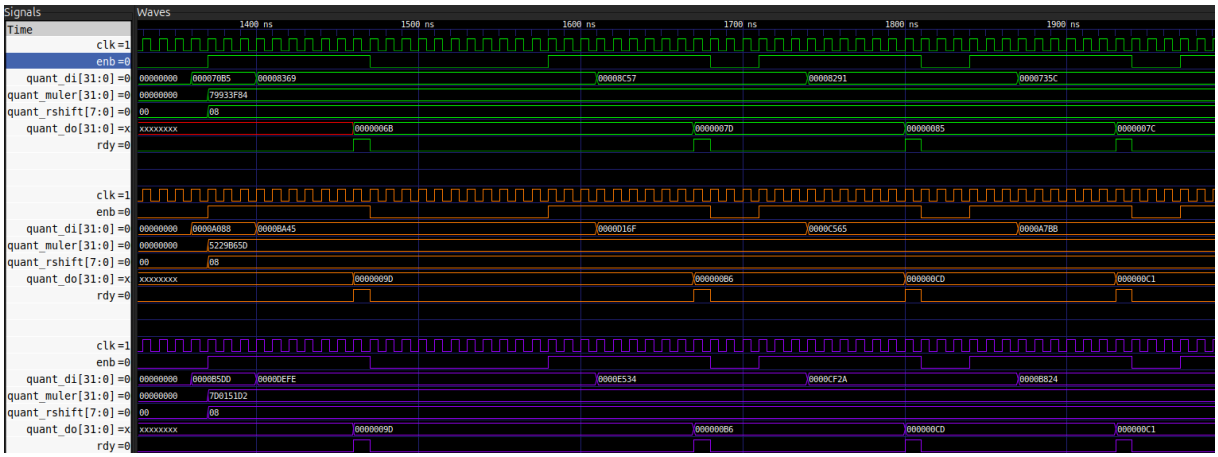
Phân tích waveform của Accumulate Matrix:

- Giai đoạn 1800 ns đến 1900 ns các giá trị *Bias* được đọc từ *Bias and Partial-Sum Buffer* và nạp vào các thanh ghi nội bộ của Accumulate Matrix.
- Giai đoạn 1930 ns đến 1940 ns sau khi toàn bộ các giá trị tổng riêng phần từ Processing Matrix đã sẵn sàng, Accumulate Matrix bắt đầu thực hiện phép cộng tích lũy giữa tổng riêng phần và *Bias*, sau đó lưu kết quả vào *Partial-Sum Register*.

– Giai đoạn 1940 ns đến 2100 ns quá trình cộng dồn được lặp lại liên tục cho các chu kỳ kế tiếp, nhằm kết hợp thêm các tổng riêng phần mới với các giá trị đã được lưu trữ, đảm bảo duy trì tính liên tục của pipeline.

3. Thực hiện tác vụ hậu xử lý POOL:

- *Quantize Unit:* Trong Flash Accelerator có 3 khối Quantize Unit đảm nhiệm việc lượng tử hóa kết quả đầu ra 32-bit từ Accumulate Matrix về dạng 8-bit, được phân biệt trong waveform bằng các màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba).
  Hình 3.7 minh họa waveform của khối Quantize Unit trong quá trình mô phỏng.



Hình 3.7: *Waveform của Quantize Unit*

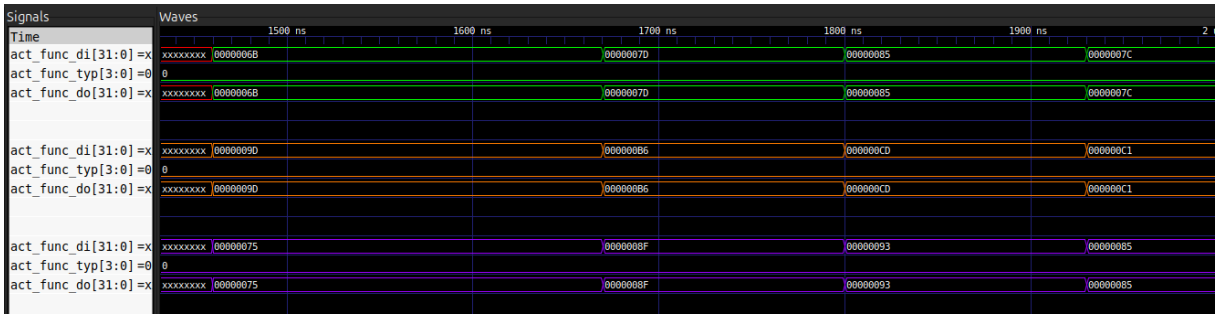
Phân tích waveform của Quantize Unit:

– Giai đoạn 1370 ns đến 1380 ns, các tham số lượng tử hóa (multiplier và shift) được nạp vào các Quantize Unit.

– Giai đoạn 1380 ns đến 1470 ns, thực hiện quá trình lượng tử hóa, chuyển đổi dữ liệu 32-bit từ Accumulate Matrix về dạng 8-bit.

– Giai đoạn 1460 ns đến 1670 ns, tiếp tục đọc dữ liệu mới từ Accumulate Matrix và thực hiện lượng tử hóa lặp lại.

- *Activation Unit:* Trong Flash Accelerator có 3 khối Activation Unit thực hiện các hàm kích hoạt phi tuyến (ReLU và ReLU6) trên dữ liệu đầu ra đã được lượng tử hóa từ Quantize Unit. Ba khối được phân biệt trong waveform bằng các màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba).
  Hình 3.8 minh họa waveform của khối Activation trong quá trình mô phỏng.

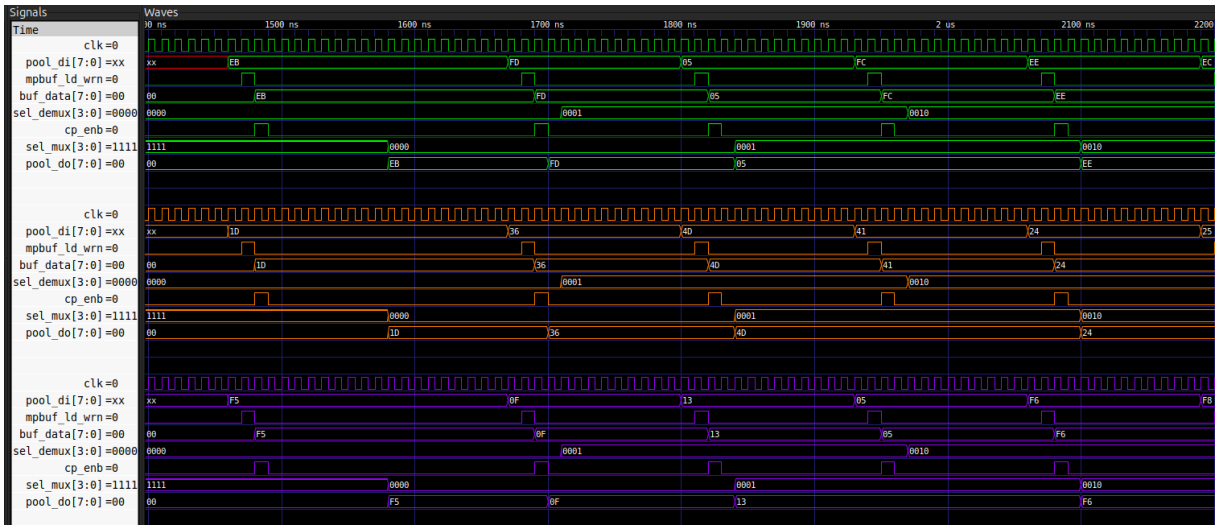


Hình 3.8: *Waveform của Activation Unit*

Phân tích waveform của Activation Unit:

- Giai đoạn 1460ns đến 1470 ns, dữ liệu 8-bit từ Quantize Unit được nạp vào các Activation Unit và bắt đầu thực hiện hàm ReLU
- Giai đoạn 1470 ns đến 1700 ns, quá trình được lặp lại.

- *Max Pooling:* Trong Flash Accelerator, khối Max Pooling chỉ được kích hoạt khi hệ thống hoạt động ở chế độ Mixed Layer. Khối này có nhiệm vụ thực hiện phép so sánh để chọn giá trị lớn nhất trong từng vùng dữ liệu đầu ra, sau khi đã đi qua bước Activation. Trong mô phỏng, ba khối Max Pooling được phân biệt trong waveform bằng các màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba).
  Hình 3.9 minh họa waveform của khối Max Pooling trong quá trình mô phỏng.



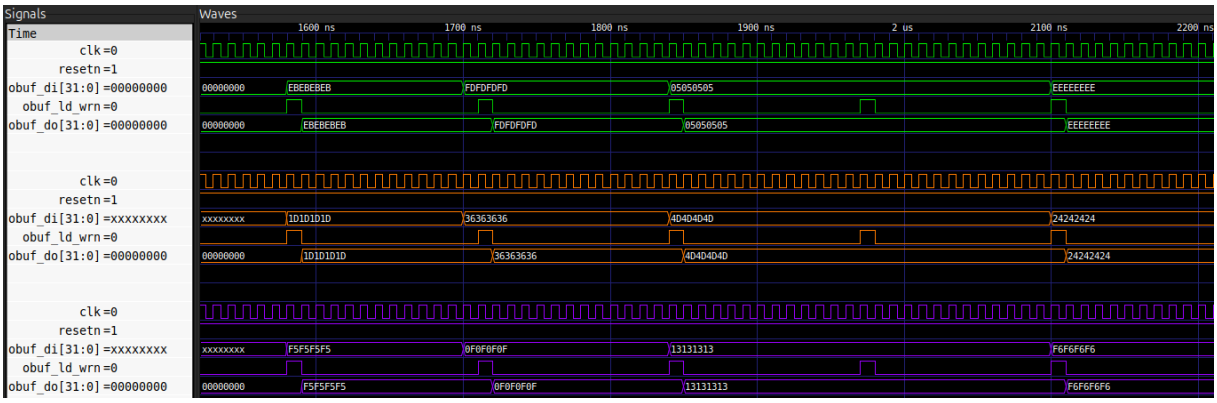
Hình 3.9: *Waveform của Max Pooling Unit*

Phân tích waveform của Max Pooling Unit:

- Giai đoạn 1470 ns đến 1480 ns, dữ liệu đầu vào từ khối Activation được nạp vào các thanh ghi đệm của Max Pooling.
- Giai đoạn 1480 ns đến 1490 ns, bộ điều khiển phân phối dữ liệu đến các khối so sánh thích hợp và tiến hành phép so sánh.
- Giai đoạn 1490 ns đến 1580 ns, kết quả so sánh được xử lý, bộ điều khiển lựa chọn giá trị lớn nhất để đưa ra đầu ra tạm thời.
- Giai đoạn 1580 ns đến 1680 ns, quá trình trên được lặp lại cho các nhóm dữ liệu tiếp theo.

4. Thực hiện tác vụ ghi dữ liệu WBACK:

1. *Output Buffer:* Trong Flash Accelerator có 3 khối Output Buffer, mỗi khối phụ trách lưu trữ một giá trị đầu ra từ COMPS hoặc POOL, có thể là tổng riêng phần 32-bit hoặc kết quả cuối cùng 8-bit sau khi qua các khối hậu xử lý. Trong waveform, các Output Buffer được phân biệt bằng ba màu: xanh lá (khối thứ nhất), cam (khối thứ hai) và tím (khối thứ ba).
   Hình 3.10 minh họa waveform của Output Buffer trong quá trình mô phỏng.



Hình 3.10: *Waveform của Output Buffer*

Phân tích waveform của Output Buffer:

- Giai đoạn 1580 ns đến 1590 ns, dữ liệu đầu ra từ COMPS hoặc POOL (trong trường hợp này là từ POOL) được đưa vào các Output Buffer thông qua khối MUX đầu vào.
- Giai đoạn 1590 ns đến 1710 ns, lưu dữ liệu trong thanh ghi.
- Giai đoạn 1710 ns đến 1840 ns, quá trình được lặp lại cho các nhóm dữ liệu tiếp theo.

3.2.2 Kết quả thực hiện

Quá trình thiết kế và triển khai Flash Accelerator được tiến hành trên công cụ **Xilinx Vivado 2025.1**, với phần cứng mục tiêu là Zynq UltraScale+ MPSoC ZCU106. Sau khi hoàn tất toàn bộ các bước *Elaboration*, *Synthesis* và *Implementation*, tần số tối đa mà hệ thống có thể đạt được là 150 MHz. Trong các thí nghiệm kiểm chứng, bộ tăng tốc được cấu hình chạy ở mức 100 MHz để đảm bảo tính ổn định. Kết quả báo cáo thời gian (*Timing Report*) của Flash Accelerator khi thực hiện trên **Vivado 2025.1** với hai tần số mục tiêu 100 MHz và 150 MHz được thể hiện trong Hình 3.11 và Hình 3.12.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	1.942 ns	Worst Hold Slack (WHS):	0.009 ns	Worst Pulse Width Slack (WPWS):	4.725 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	10903	Total Number of Endpoints:	10903	Total Number of Endpoints:	4964

All user specified timing constraints are met.

Hình 3.11: *Kết quả báo cáo về thời gian của khối Flash Accelerator khi chạy với tần số 100Mhz*

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	0.281 ns	Worst Hold Slack (WHS):	0.009 ns	Worst Pulse Width Slack (WPWS):	3.058 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	10903	Total Number of Endpoints:	10903	Total Number of Endpoints:	4958

All user specified timing constraints are met.

Hình 3.12: *Kết quả báo cáo về thời gian của khối Flash Accelerator khi chạy với tần số 150Mhz*

Nhận xét và phân tích:

- **Worst Negative Slack (WNS):** Đây là chỉ số quan trọng nhất phản ánh khả năng đáp ứng ràng buộc *setup time*.
  - Ở 100 MHz, giá trị WNS đạt 1.942 ns, cho thấy thiết kế dư nhiều biên an toàn, có thể hoạt động ổn định mà không vi phạm ràng buộc thời gian.
  - Ở 150 MHz, WNS giảm xuống còn 0.281 ns, tức là biên an toàn rất nhỏ. Mặc dù vẫn thỏa mãn yêu cầu thời gian (WNS > 0), hệ thống đã tiến gần tới giới hạn tốc độ cực đại.
- **Worst Hold Slack (WHS):** Giá trị WHS của hai tần số đều bằng 0.009 ns (150 MHz), cả hai đều dương, chứng tỏ hệ thống không gặp vấn đề về *hold time*.
- **Worst Pulse Width Slack (WPWS):** WPWS có giá trị 4.725 ns ở 100 MHz và giảm còn 3.058 ns ở 150 MHz. Đây là mức chênh lệch bình thường khi tăng tần số, và vẫn đảm bảo an toàn.
- **Khả năng mở rộng tần số:** Với kết quả trên, tần số 100 MHz là mức an toàn nhất cho hoạt động lâu dài. Ở 150 MHz, hệ thống vẫn chạy được nhưng biên thời gian chỉ còn rất nhỏ (0.281 ns). Nếu tiếp tục tăng tần số (ví dụ lên 160–170 MHz), khả năng cao là WNS sẽ âm, dẫn tới vi phạm ràng buộc *setup time* và hệ thống mất ổn định.

Ngoài kết quả về thời gian, các kết quả dưới đây thu được khi hiện thực Flash Accelerator ở tần số 100Mhz.

1. **Về kết quả tài nguyên sử dụng:**
   - Kết quả tổng quát:

Tài nguyên	Sử dụng	Có sẵn	Tỉ lệ (%)
LUT	12,315	230,400	5.35
FF	5,059	460,800	1.1
DSP	162	1,728	9.38
BRAM	0	312	0

Bảng 3.2: *Báo cáo sử dụng tài nguyên của Flash Accelerator*

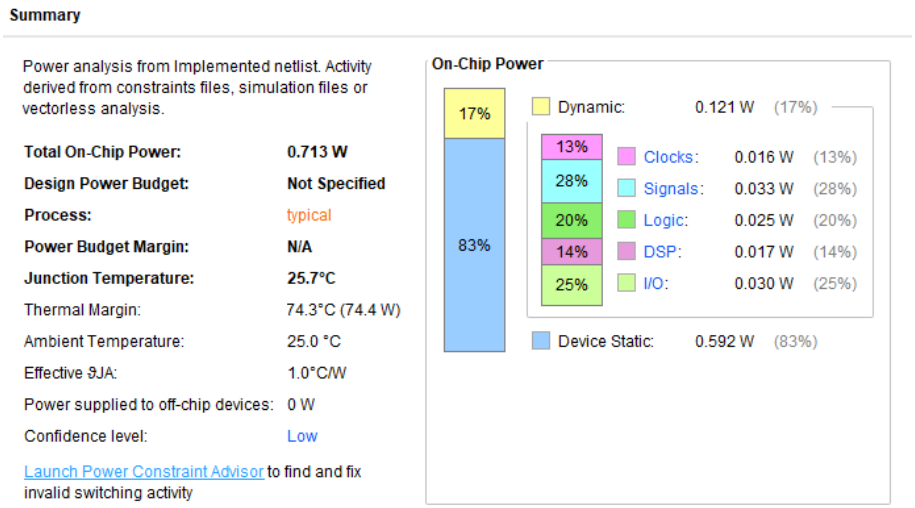
   Nhận xét: Kết quả báo cáo cho thấy bộ tăng tốc Flash Accelerator chỉ chiếm một phần rất nhỏ so với tổng tài nguyên khả dụng trên FPGA ZCU106. Cụ thể, số lượng LUT và FF sử dụng lần lượt chiếm khoảng 5.36% và 1.1%, trong khi số lượng DSP Slice chiếm 9.38%. Điều này chứng tỏ thiết kế có mức độ tối ưu cao, vẫn còn nhiều tài nguyên dự phòng để mở rộng quy mô hoặc tích hợp thêm các chức năng khác.
   - Kết quả cụ thể từng khối:

   Các khối không được liệt kê riêng trong bảng thực chất chỉ sinh ra các phần tử *register inference* (flip-flop hoặc LUTRAM nhỏ). Trong quá trình tổng hợp, Vivado đã tối ưu và gộp các tài nguyên này trực tiếp vào khối cha *fs_ accel*, thay vì hiển thị dưới dạng các khối độc lập.

2. **Về năng lượng tiêu thụ:**

Tên	CLB LUTs		CLB Registers		CLB	
fs_accel	12,315	100%	5,059	100%	2,632	100%
flow_ctrl	5,720	46.4%	670	13.3%	1,373	52.2%
config_regs	3,301	26.8%	1,876	37.1%	1,039	39.5%
pu_matrix	1,895	15.4%	506	10.0%	429	16.3%
pool_unit	976	7.9%	825	16.3%	365	13.9%
acc_matrix	111	0.9%	288	5.7%	84	3.2%
wbuf_0	0	0.0%	72	1.4%	23	0.9%
wbuf_1	0	0.0%	72	1.4%	19	0.7%
wbuf_2	0	0.0%	72	1.4%	18	0.7%
ibuf_0	71	0.6%	162	3.2%	81	3.1%
ibuf_1	85	0.7%	162	3.2%	81	3.1%
ibuf_2	68	0.6%	162	3.2%	86	3.3%

Bảng 3.3: *Báo cáo chi tiết sử dụng tài nguyên của các khối trong Flash Accelerator*



Hình 3.13: *Kết quả báo cáo về năng lượng tổng quát của Flash Accelerator*

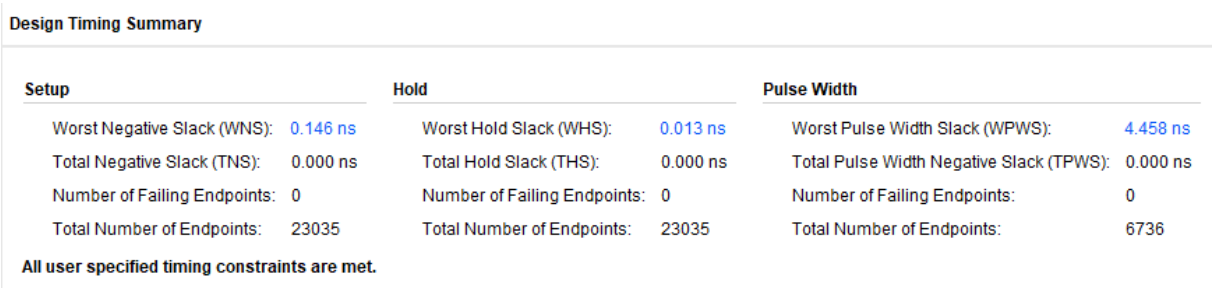
Tổng công suất tiêu thụ của Flash Accelerator đạt 0.714 W, trong đó công suất tĩnh chiếm ưu thế với 0.592 W (83%), so với công suất động chỉ 0.122 W (17%). Kết quả này cho thấy thiết kế có mức tiêu thụ động thấp, chủ yếu phát sinh từ hoạt động truyền dữ liệu, và bảo đảm an toàn nhiệt khi vận hành ở tần số 100 MHz. Lưu ý rằng các giá trị trên chỉ mang tính tham khảo và được ước lượng bởi công cụ Vivado.

3.3 Hiện thực tích hợp Flash Accelerator với khối điều khiển PicoSoC

Flash Accelerator được tích hợp cùng khối điều khiển PicoSoC, tạo thành hệ thống FSPicoSoC, với mục tiêu tăng tốc quá trình tính toán các layer trong mạng CNN. Kiến trúc tích hợp này được xây dựng dựa trên mô tả chi tiết tại Mục 2.1.

Hệ thống được hiện thực phần cứng với bộ nhớ lệnh (*Instruction Memory – IMEM*) dung lượng 256 KB và bộ nhớ dữ liệu (*Data Memory – DMEM*) dung lượng 128 KB. Thiết kế hoạt động ổn định ở tần số 100 MHz, được triển khai trên công cụ **Xilinx Vivado 2025.1** với *target board* là **Zynq UltraScale+ MPSoC ZCU106**.

Kết quả báo cáo thời gian cho thấy hệ thống FSPicoSoC đáp ứng đầy đủ các ràng buộc timing ở tần số 100 MHz. Các giá trị Worst Negative Slack (WNS = 0.146 ns) và Worst Hold Slack (WHS = 0.013 ns) đều dương, cho thấy thiết kế không có điểm lỗi về timing. Điều này khẳng định hệ thống hoạt động ổn định và phù hợp để triển khai trên phần cứng thực tế.



Hình 3.14: *Kết quả báo cáo về thời gian của FSPicoSoC khi chạy với tần số 100Mhz*

1. Về kết quả tài nguyên sử dụng:

- Kết quả tổng quát:

Tài nguyên	Sử dụng	Có sẵn	Tỉ lệ (%)
LUT	15,418	230,400	6.69
FF	6,391	460,800	1.39
DSP	162	1,728	9.38
BRAM	180	312	57.69

Bảng 3.4: *Báo cáo sử dụng tài nguyên của FSPicoSoC*

Báo cáo sử dụng tài nguyên cho thấy FSPicoSoC chỉ chiếm một tỷ lệ nhỏ LUT (6.59%) và FF (1.39%), trong khi mức sử dụng DSP (9.38%) vẫn còn thấp so với tài nguyên sẵn có. Điểm đáng chú ý là BRAM chiếm 57.69%, phản ánh nhu cầu bộ nhớ lớn của hệ thống, nhưng vẫn nằm trong giới hạn cho phép của ZCU106.

- Kết quả cụ thể từng khối:

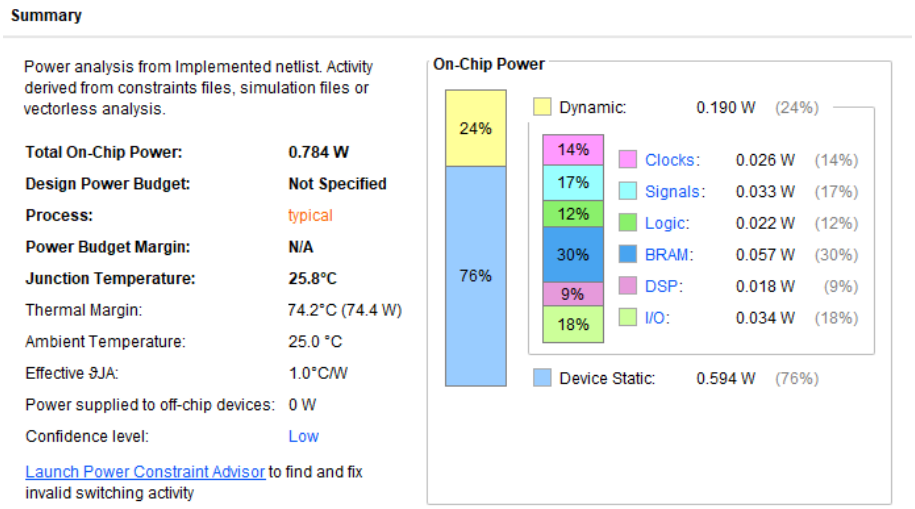
Tên	CLB LUTs		CLB Registers		CLB		Block RAM	
fs_picosoc	15,418	100%	6,391	100%	1,156	100%	180	100%
accelerator	12,410	80.5%	5,054	79.1%	954	82.5%	0	0%
cpu	2,355	15.3%	1,188	18.6%	498	43.1%	0	0%
dmem	84	0.5%	1	0.02%	53	4.6%	64	35.6%
imem	233	1.5%	3	0.05%	106	9.2%	116	64.4%
simpleuart	91	0.6%	131	2.1%	31	2.7%	0	0%

Bảng 3.5: *Báo cáo chi tiết sử dụng tài nguyên của các khối trong FSPicoSoC*

Kết quả cho thấy khối *accelerator* chiếm tỷ lệ tài nguyên lớn nhất (với hơn 80% LUT và gần 79% FF). Toàn bộ *Block RAM* chủ yếu được phân bổ cho *IMEM* và *DMEM*, cho thấy bộ nhớ là thành phần chiếm ưu thế trong hệ thống. Các khối phụ trợ như *simpleuart* tiêu tốn rất ít tài nguyên và hầu như không ảnh hưởng đến tổng thể.

2. Về năng lượng tiêu thụ:

Tổng công suất của FSPicoSoC đạt 0.784 W, cao hơn so với khi chỉ chạy riêng Flash Accelerator (0.714 W). Công suất động tăng lên 0.190 W (24%), chủ yếu do hoạt động bổ sung từ CPU, bộ nhớ và giao tiếp. Trong khi đó, công suất tĩnh vẫn chiếm ưu thế (76%), cho thấy hệ thống mở rộng thêm PicoSoC không làm thay đổi đáng kể đặc tính năng lượng, nhưng có mức tiêu thụ cao hơn do tải phần cứng phức tạp hơn.



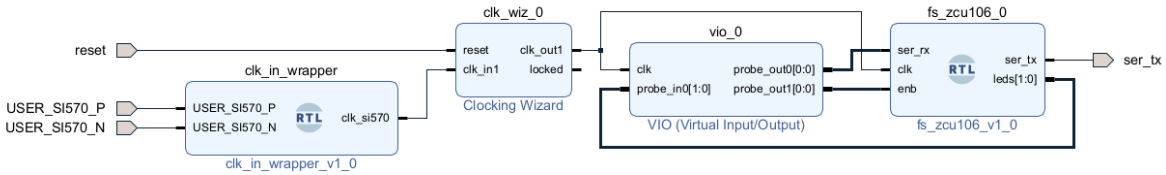
Hình 3.15: *Kết quả báo cáo về năng lượng tổng quát của FSPicoSoC*

3.4 Triển khai và đánh giá trên Zynq UltraScale+ MPSoC ZCU106 FPGA

Trong hệ thống này, người dùng không truy cập trực tiếp đến board FPGA từ máy cá nhân, mà thông qua một Server PC trung gian. Kết nối tới server được thực hiện bằng Tailscale, cho phép tạo mạng riêng ảo (VPN) bảo mật và truy cập từ xa ở bất kỳ đâu. Sau khi kết nối vào server, người dùng sử dụng giao thức SSH để điều khiển board ZCU106 FPGA gián tiếp, đồng thời có thể sử dụng các công cụ như Vivado và VIO để nạp bitstream và giám sát hoạt động của thiết kế.

Đối với việc nạp chương trình, hệ thống sử dụng cơ chế *$readmemh* để ánh xạ trực tiếp nội dung firmware (*.mem*) vào Block RAM (BRAM) ngay từ giai đoạn tổng hợp. Cách tiếp cận này giúp CPU trong PicoSoC có thể truy cập lệnh ngay sau khi reset mà không cần tới bộ xử lý ARM trong *Processing System (PS)*. Tuy nhiên, hạn chế lớn của phương pháp này là tính linh hoạt: mỗi lần thay đổi firmware đều đòi hỏi tái tổng hợp và hiện thực lại thiết kế, gây tốn thời gian khi phát triển.

Từ các lý thuyết trên ta xây dựng một Block Design như Hình 3.16:



Hình 3.16: *Mô hình Block Design thực hiện trên Zynq UltraScale+ MPSoC ZCU106*

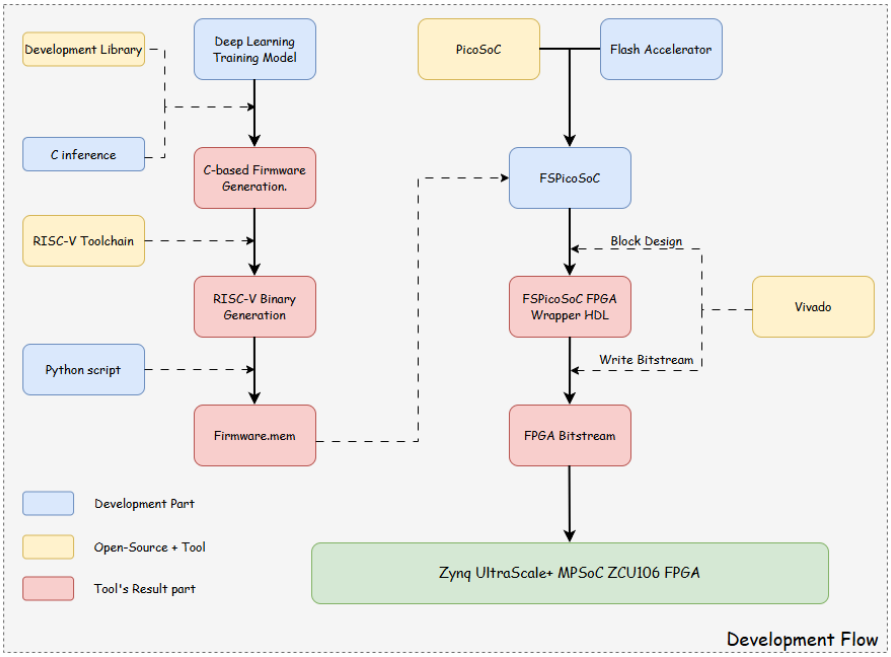
Các thành phần chính trong Block Design bao gồm:

- **clk_in_wrapper:** Nhận xung clock từ board ZCU106 thông qua hai chân `USER_SI570_P/N`. Đây là nguồn clock gốc do board sinh ra với tần số 150 MHz, được đưa vào hệ thống như clock tham chiếu.
- **Clocking Wizard:** Biến đổi xung clock 150 MHz từ clk_in_wrapper thành xung 100 MHz, đảm bảo khối `fs_zcu106` hoạt động ổn định. Ngoài ra, Clocking Wizard cũng phát tín hiệu locked để đồng bộ reset, giúp hệ thống chỉ khởi chạy khi nguồn clock đã ổn định.
- **VIO (Virtual Input/Output):** Được sử dụng như cầu nối debug, cho phép giám sát và điều

khiển các tín hiệu điều khiển (enb) cũng như quan sát trạng thái đầu ra (leds) trực tiếp trên Vivado Hardware Manager, hỗ trợ kiểm thử mà không cần ngoại vi bổ sung.

- **fs_zcu106:** Khối RTL chính chứa Flash Accelerator, thực hiện tính toán và xử lý dữ liệu. Khối này nhận clock 100 MHz từ Clocking Wizard, tín hiệu điều khiển từ VIO, và giao tiếp dữ liệu qua UART (ser_rx, ser_tx) để truyền/nhận kết quả.

Nhờ cấu hình block design này, việc nạp bitstream, điều khiển enable/disable, và quan sát kết quả qua UART có thể được thực hiện trực tiếp trên board ZCU106. Cách tiếp cận này vừa đảm bảo kiểm thử nhanh chóng, vừa hỗ trợ debug linh hoạt thông qua công cụ VIO của Vivado.
Dưới đây sẽ là quy trình thiết kế phần cứng và cách thực hiện trên board ZCU106:



Hình 3.17: *Quy trình thiết kế để chạy trên board zcu106*

3.4.1 Kết quả kiểm thử ở cấp Layer

3.4.1.1 Convolution Layer

A. Mô tả

Trong phần đánh giá, tổng cộng 7 test case được xây dựng để kiểm thử tốc độ tính toán ở lớp Convolution. Các cấu hình cụ thể của từng test case được trình bày trong Bảng 3.6.

Kiểm thử	Input Feature Map	Kernel	Stride	Output Feature Map	Bias
Test Case 0	$7 \times 7 \times 3$	$3 \times 3 \times 3 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 1	$7 \times 7 \times 6$	$3 \times 3 \times 6 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 2	$7 \times 7 \times 9$	$3 \times 3 \times 9 \times 9$	1×1	$5 \times 5 \times 9$	9
Test Case 3	$13 \times 13 \times 3$	$3 \times 3 \times 3 \times 6$	1×1	$11 \times 11 \times 6$	6
Test Case 4	$13 \times 13 \times 6$	$3 \times 3 \times 6 \times 6$	1×1	$11 \times 11 \times 6$	6
Test Case 5	$13 \times 13 \times 9$	$3 \times 3 \times 9 \times 9$	1×1	$11 \times 11 \times 9$	9
Test Case 6	$28 \times 28 \times 3$	$3 \times 3 \times 3 \times 32$	1×1	$26 \times 26 \times 32$	32

Bảng 3.6: *Các test case dùng để đánh giá hoạt động của Convolution Layer*

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Convolution Layer trên hệ thống PicoSoC được trình bày trong Bảng 3.7.

Kiểm thử	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x0008b82b	571,435	0x0001c0ca	114,890
Test Case 1	0x00103975	1,063,285	0x00034382	213,890
Test Case 2	0x002393b6	2,331,574	0x00072878	469,112
Test Case 3	0x002a27d3	2,762,707	0x00087a0a	555,530
Test Case 4	0x004e7ab6	5,143,222	0x000fc9c1	1,034,689
Test Case 5	0x00aa9ab7	11,180,727	0x0022a31b	2,269,979
Test Case 6	0x04da085d	81,397,853	0x00fbf82e	16,513,070

Bảng 3.7: *Kết quả thực thi các test case Convolution Layer trên hệ thống PicoSoC*

C. Hệ thống FSPicoSoC

Kết quả triển khai các test case của Convolution Layer trên hệ thống FSPicoSoC được trình bày trong Bảng 3.8.

Kiểm thử	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x0000036f	879	0x00000080	128
Test Case 1	0x000006e1	1,761	0x000000fe	254
Test Case 2	0x00000fd9	4,057	0x00000246	582
Test Case 3	0x00000c05	3,077	0x000001ba	442
Test Case 4	0x00001876	6,262	0x00000382	898
Test Case 5	0x00003a65	14,949	0x0000085a	2,138
Test Case 6	0x00018ab9	101,049	0x00003866	14,438

Bảng 3.8: *Kết quả thực thi các test case Convolution Layer trên hệ thống FSPicoSoC*

D. Đánh giá

Test Case	Số chu kỳ			Số lệnh thực thi		
	PicoSoC	FSPicoSoC	Tỉ lệ	PicoSoC	FSPicoSoC	Tỉ lệ
0	571,435	879	650	114,890	128	897
1	1,063,285	1,761	604	213,890	254	842
2	2,331,574	4,057	574	469,112	582	806
3	2,762,707	3,077	898	555,530	442	1,257
4	5,143,222	6,262	821	1,034,689	898	1,152
5	11,180,727	14,949	748	2,269,979	2,138	1,061
6	81,397,853	101,049	805	16,513,070	14,438	1,143

Bảng 3.9: *So sánh số chu kỳ và số lệnh thực thi giữa PicoSoC và FSPicoSoC ở Convolution Layer*

Bảng 3.9 cho thấy FSPicoSoC giúp giảm mạnh cả số chu kỳ lẫn số lệnh thực thi so với PicoSoC. Với chu kỳ, mức tăng tốc đạt từ 574 lần đến 898 lần, trong khi với số lệnh thực thi, mức tăng tốc còn ấn tượng hơn, dao động từ 806 lần đến 1,257 lần. Điều này khẳng định rằng việc tích hợp Flash Accelerator không chỉ tăng hiệu năng xử lý mà còn tối ưu hoá số lệnh, giúp hệ thống hoạt động hiệu quả hơn đáng kể.

3.4.1.2 Mixed Layer

A. Mô tả

Trong phần đánh giá, tổng cộng 7 test case được xây dựng để kiểm thử tốc độ tính toán ở lớp Mixed (bao gồm Convolution Layer với Max Pooling Layer). Các cấu hình cụ thể của từng test case được trình bày trong Bảng 3.10.

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Mixed Layer trên hệ thống PicoSoC được trình bày trong Bảng 3.11.

Kiểm thử	Input Feature Map	Kernel	Stride	Output Feature Map	Bias
Test Case 0	$7 \times 7 \times 3$	$3 \times 3 \times 3 \times 6$	1×1	$2 \times 2 \times 6$	6
Test Case 1	$7 \times 7 \times 6$	$3 \times 3 \times 6 \times 6$	1×1	$2 \times 2 \times 6$	6
Test Case 2	$7 \times 7 \times 9$	$3 \times 3 \times 9 \times 9$	1×1	$2 \times 2 \times 9$	9
Test Case 3	$13 \times 13 \times 3$	$3 \times 3 \times 3 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 4	$13 \times 13 \times 6$	$3 \times 3 \times 6 \times 6$	1×1	$5 \times 5 \times 6$	6
Test Case 5	$13 \times 13 \times 9$	$3 \times 3 \times 9 \times 9$	1×1	$5 \times 5 \times 9$	9
Test Case 6	$28 \times 28 \times 3$	$3 \times 3 \times 3 \times 32$	1×1	$13 \times 13 \times 32$	32

Bảng 3.10: *Các test case dùng để đánh giá hoạt động của Mixed Layer*

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x0008cd46	576,838	0x0001c8f2	116,978
Test Case 1	0x00103d29	1,064,233	0x00034ba1	215,969
Test Case 2	0x00237d52	2,325,842	0x00073446	472,134
Test Case 3	0x002abfa0	2,801,568	0x0008aaae	567,982
Test Case 4	0x004ebd6c	5,160,300	0x000ffa75	1,047,157
Test Case 5	0x00ad9862	11,376,738	0x0022ead2	2,288,338
Test Case 6	0x0506e533	84,337,971	0x010269fe	16,935,422

Bảng 3.11: *Kết quả thực thi các test case Mixed Layer trên hệ thống PicoSoC*

C. Hệ thống FSPicoSoC

Kết quả triển khai các test case của Mixed Layer trên hệ thống FSPicoSoC được trình bày trong Bảng 3.12.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x00000392	914	0x00000086	134
Test Case 1	0x000006da	1,754	0x000000fe	254
Test Case 2	0x00000fc4	4,036	0x00000244	580
Test Case 3	0x00000d4e	3,406	0x000001ea	490
Test Case 4	0x00001868	6,248	0x00000380	896
Test Case 5	0x00003a57	14,935	0x00000858	2,136
Test Case 6	0x0001b537	111,927	0x00003e78	15,992

Bảng 3.12: *Kết quả thực thi các test case Mixed Layer trên hệ thống FSPicoSoC*

D. Đánh giá

Test Case	Số chu kỳ			Số lệnh thực thi		
	PicoSoC	FSPicoSoC	Tỉ lệ	PicoSoC	FSPicoSoC	Tỉ lệ
0	576,838	914	631	116,978	134	873
1	1,064,233	1,754	607	215,969	254	850
2	2,325,842	4,036	576	472,134	580	814
3	2,801,568	3,406	822	567,982	490	1,159
4	5,160,300	6,248	826	1,047,157	896	1,168
5	11,376,738	14,935	762	2,288,338	2,136	1,071
6	84,337,971	111,927	753	16,935,422	15,992	1,059

Bảng 3.13: *So sánh số chu kỳ và số lệnh thực thi giữa PicoSoC và FSPicoSoC ở Mixed Layer*

Bảng 3.13 cho thấy FSPicoSoC mang lại hiệu năng vượt trội so với PicoSoC trong Mixed Layer. Về số chu kỳ, mức tăng tốc đạt từ 576 lần đến 826 lần, trong khi với số lệnh thực thi, mức tăng tốc cao hơn, từ 814 lần đến 1,168 lần. Điều này chứng tỏ Flash Accelerator không chỉ rút ngắn thời gian xử lý mà còn tối ưu số lệnh thực thi, đặc biệt hiệu quả ở các test case có cấu hình lớn (Testcase 5, Testcase 6).

3.4.1.3 Fully-Connected Layer

A. Mô tả

Trong phần đánh giá, tổng cộng 6 test case được xây dựng để kiểm thử tốc độ tính toán ở lớp Fully-Connected. Các cấu hình cụ thể của từng test case được trình bày trong Bảng 3.14.

Tên	Input Activation	Weight	Output Activation	Bias
Test Case 0	36	36×9	9	9
Test Case 1	36	36×27	27	27
Test Case 2	54	54×9	9	9
Test Case 3	54	54×27	27	27
Test Case 4	162	162×9	9	9
Test Case 5	162	162×27	27	27

Bảng 3.14: *Các test case dùng để đánh giá hoạt động của Fully-Connected Layer*

B. Hệ thống PicoSoC

Kết quả triển khai các test case của Fully-Connected Layer trên hệ thống PicoSoC được trình bày trong Bảng 3.15.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x000056e6	22,246	0x00000fef	4,079
Test Case 1	0x000101f8	66,040	0x00002f51	12,113
Test Case 2	0x00007a56	31,318	0x00001643	5,699
Test Case 3	0x00016c48	93,256	0x00004251	16,977
Test Case 4	0x00014ef6	85,750	0x00003c3a	15,418
Test Case 5	0x0003ea28	256,552	0x0000b435	46,133

Bảng 3.15: *Kết quả thực thi các test case Fully-Connected Layer trên hệ thống PicoSoC*

C. Hệ thống FSPicoSoC

Kết quả triển khai các test case của Fully-Connected Layer trên hệ thống FSPicoSoC được trình bày trong Bảng 3.16.

Tên	Số chu kỳ		Số lệnh thực thi	
	Hex	Dec	Hex	Dec
Test Case 0	0x000001d2	466	0x00000046	70
Test Case 1	0x0000048e	1166	0x000000aa	170
Test Case 2	0x00000296	662	0x00000062	98
Test Case 3	0x000006b0	1712	0x000000f8	248
Test Case 4	0x00000712	1810	0x00000106	262
Test Case 5	0x00001352	4946	0x000002c6	710

Bảng 3.16: *Kết quả thực thi các test case Fully-Connected Layer trên hệ thống FSPicoSoC*

D. Đánh giá

Test Case	Số chu kỳ			Số lệnh thực thi		
	PicoSoC	FSPicoSoC	Tỉ lệ	PicoSoC	FSPicoSoC	Tỉ lệ
0	22,246	466	48	4,079	70	58
1	66,040	1,166	57	12,113	170	71
2	31,318	662	47	5,699	98	58
3	93,256	1,712	54	16,977	248	68
4	85,750	1,810	47	15,418	262	59
5	256,552	4,946	52	46,133	710	65

Bảng 3.17: *So sánh số chu kỳ và số lệnh thực thi giữa PicoSoC và FSPicoSoC ở Fully-Connected Layer*

Bảng 3.17 cho thấy FSPicoSoC cải thiện hiệu năng rõ rệt so với PicoSoC trong Fully-Connected Layer. Về số chu kỳ, mức tăng tốc dao động từ 47 lần đến 56 lần, trong khi với số lệnh thực thi, mức tăng tốc cao hơn, từ 57 lần đến 65 lần. Kết quả này chứng minh Flash Accelerator giúp rút ngắn thời gian tính toán và giảm đáng kể số lệnh phải thực thi.

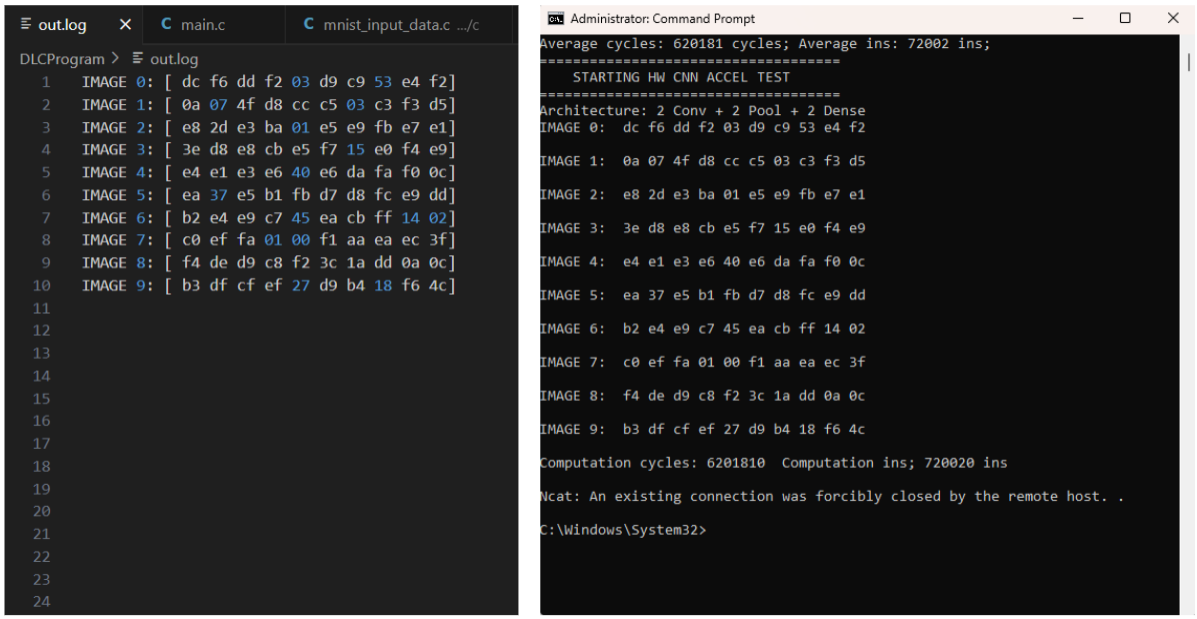
3.4.2 Kết quả kiểm thử ở cấp Model

A. Mô tả Model

Model được sử dụng trong phần kiểm thử này đã được trình bày chi tiết ở Phần 3.1. Đây là một mạng nơ-ron gồm 6 tầng, được thiết kế theo cấu trúc đơn giản nhưng đủ để kiểm chứng hiệu năng hệ thống. Cụ thể, mô hình bao gồm 2 tầng Convolution kết hợp Pooling (Mixed Layer) và 2 tầng Fully-Connected Layer, hay còn được gọi tắt là 2 Mixed + 2 Fully. Thiết kế này vừa đảm bảo tính gọn nhẹ, vừa phản ánh đặc trưng tính toán phổ biến của các mô hình CNN.

B. Kết quả kiểm thử

Độ đúng đắn chức năng của mô hình được triển khai trên FSPicoSoC được kiểm chứng bằng cách so sánh kết quả suy luận trên 10 mẫu dữ liệu MNIST thử nghiệm với golden model (mô hình tham chiếu viết bằng ngôn ngữ C). Kết quả cho thấy các vector đầu ra sinh ra bởi phần cứng trùng khớp hoàn toàn với kết quả từ golden model ở mọi mẫu kiểm thử. Điều này khẳng định rằng FSPicoSoC đã thực hiện chính xác các phép toán neural cơ bản, không phát sinh sai lệch số học so với mô hình tham chiếu.



Hình 3.18: *So sánh kết quả suy luận giữa Golden Model (bên trái) và FSPicoSoC (bên phải)*

D. Đánh giá

Kết quả triển khai Model lên hệ thống FSPicoSoC trên ZCU106 như sau:

- **Độ chính xác (Accuracy):** Mô hình đạt độ chính xác 98.3% khi kiểm thử trên 1000 mẫu dữ liệu từ tập kiểm thử MNIST.
- **Hiệu năng suy luận (Inference Performance):** Tốc độ thực hiện tác vụ suy luận được đánh giá trên 10 mẫu dữ liệu từ tập kiểm thử MNIST, kết quả chi tiết sẽ được phân tích trong các Bảng 3.18.

Để làm rõ hiệu quả tăng tốc, Bảng 3.18 trình bày so sánh kết quả thực thi toàn bộ mô hình trên PicoSoC[?] gốc và FSPicoSoC.

Tác vụ	PicoSoC[?]		FSPicoSoC		Tăng tốc	
	Chu kỳ	Lệnh	Chu kỳ	Lệnh	Chu kỳ	Lệnh
Đọc dữ liệu	2,409,948	217,390	333,050	65,160	7.24	3.34
Tính toán	7,277,166,368	1,171,320,094	5,934,550	842,060	1226	1390
In kết quả	6,337,480	22,360	4,880,150	11,740	1.30	1.90
Tổng cộng	7,285,913,796	1,171,559,844	11,355,540	919,720	641	1274

Bảng 3.18: *So sánh kết quả thực thi Model giữa PicoSoC và FSPicoSoC*

Bảng 3.18 cho thấy sự khác biệt rõ rệt ở tác vụ tính toán. FSPicoSoC đạt mức tăng tốc khoảng 1226 lần về chu kỳ và 1390 lần về số lệnh thực thi so với PicoSoC. Điều này chứng minh rằng việc tích hợp Flash Accelerator đã giúp giảm tải hầu hết các phép nhân–cộng tích luỹ (MAC) từ CPU sang phần cứng chuyên dụng, qua đó mang lại hiệu năng vượt trội trong xử lý các tầng CNN.

Ngược lại, các tác vụ đọc dữ liệu và in kết quả chỉ cải thiện ở mức giới hạn (7.24 lần và 1.30 lần về chu kỳ), nguyên nhân là do các tác vụ này phụ thuộc nhiều vào cơ chế giao tiếp ngoại vi (UART, bộ nhớ), không nằm trong phạm vi tăng tốc chính của hệ thống. Do đó, các số liệu đọc/ghi chỉ mang tính tham khảo, trong khi trọng tâm nghiên cứu tập trung vào hiệu quả tăng tốc ở khối tính toán.

3.5 Đánh giá kết quả và so sánh với các nghiên cứu liên quan

3.5.1 Đánh giá kết quả

3.5.1.1 Tổng hợp về tài nguyên và năng lượng

So sánh về tài nguyên

Tiêu chí	PicoSoC	FSPicoSoC	Thay đổi
LUTs	2,767	15,418	+12,651 (+456.9%)
FFs	1,193	6,391	+5,198 (+435.6%)
DSPs	4	162	+158 (40.5%)
BRAMs	180	180	0 (Không đổi)

Bảng 3.19: *So sánh tài nguyên giữa PicoSoC và FSPicoSoC.*

Nhận xét: Sau khi tích hợp Flash Accelerator, số lượng LUTs và FFs tăng lần lượt khoảng 4.6 và 4.3 lần, phản ánh sự bổ sung logic điều khiển và đường dữ liệu mới. Số lượng DSPs tăng mạnh (40.5 lần) do toàn bộ khối MAC được phần cứng hóa trong Accelerator. Trong khi đó, BRAMs giữ nguyên ở mức 180, do cả hai hệ thống đều sử dụng cùng dung lượng bộ nhớ. Mặc dù tài nguyên tăng đáng kể, mức sử dụng này vẫn nằm trong giới hạn cho phép của FPGA ZCU106, chứng tỏ thiết kế có tính khả thi cao.

So sánh năng lượng

Nhận xét: Tổng công suất của hệ thống tăng nhẹ +0.051 W (tương ứng +6.96%), chủ yếu đến từ công suất động. Cụ thể, mức tiêu thụ của Clocks, Signals, Logic và đặc biệt DSPs tăng mạnh, phản ánh việc bổ sung khối xử lý song song trong Accelerator. Trong khi đó, BRAM giữ nguyên và Static Power không thay đổi đáng kể, còn I/O giảm nhẹ (17.1%). Nhìn chung, mức tiêu thụ năng lượng tăng nhưng vẫn ở ngưỡng thấp, đảm bảo hệ thống hoạt động ổn định ở tần số 100 MHz.

Thông số	PicoSoC	FSPicoSoC	Thay đổi
Total On-Chip	0.733	0.784	+0.051 (+6.96%)
Dynamic	0.139	0.190	+0.051 (+36.7%)
Static	0.594	0.594	+0.000 (Không đổi)
Clocks	0.012	0.026	+0.014 (2.17 lần)
Signals	0.019	0.033	+0.014 (+73.7%)
Logic	0.008	0.022	+0.014 (2.75 lần)
BRAM	0.057	0.057	+0.000 (Không đổi)
DSP	0.002	0.018	+0.016 (9 lần)

Bảng 3.20: *So sánh năng lượng giữa PicoSoC và FSPicoSoC.*

3.5.1.2 Tổng hợp về khả năng tăng tốc

Do mục tiêu của luận văn là tăng tốc suy luận CNN, phần này tập trung đánh giá hệ số tăng tốc ở cấp độ lớp và cấp độ mô hình, từ đó rút ra những đặc trưng quan trọng về hiệu quả thực thi.

A Theo lớp (layer-level).

- **Convolution Layer** (Bảng 3.9): hệ số tăng tốc đạt từ 574 lần đến 898 lần về chu kỳ, trong khi số lệnh thực thi giảm tương ứng với mức tăng tốc từ 806 lần đến 1,257 lần.
- **Mixed Layer** (Bảng 3.13): hệ số tăng tốc dao động từ 576 lần đến 826 lần về chu kỳ, và từ 814 lần đến 1,168 lần về số lệnh.
- **Fully-Connected Layer** (Bảng 3.17): tốc độ cải thiện ít hơn, với hệ số tăng tốc từ 47 lần đến 56 lần về chu kỳ và từ 57 lần đến 65 lần về số lệnh.

Nhận xét: Convolution và Mixed là những lớp có mật độ phép tính MAC cao, phù hợp để ánh xạ sang DSP trong Accelerator, nhờ đó mức tăng tốc rất lớn. Trong khi đó, Fully-Connected Layer mang tính memory-bound hơn, phụ thuộc nhiều vào băng thông đọc/ghi dữ liệu, nên hệ số tăng tốc thấp hơn nhưng vẫn duy trì hiệu quả đáng kể.

B Theo mô hình (model-level).

Phân tích tổng thể trên toàn mô hình CNN gồm 6 tầng với cấu trúc 2 Convolution + 2 Pooling + 2 Fully-Connected (xem Bảng 3.18) cho thấy phần Computation là vấn đề chính. Khi tích hợp Accelerator, FSPicoSoC đạt mức tăng tốc 1226 lần về chu kỳ và 1390 lần về số lệnh trong tác vụ tính toán so với PicoSoC. Điều này khẳng định rằng việc chuyển toàn bộ phép nhân–cộng tích lũy (MAC) sang phần cứng song song đã mang lại lợi ích rất lớn.
Ngược lại, các tác vụ đọc dữ liệu và in kết quả (thông qua UART) chỉ mang tính chất tham khảo bởi đây là các tác vụ phụ thuộc ngoại vi, không nằm trong phạm vi tăng tốc.

C Kết luận tổng quan về hiệu năng.

- FSPicoSoC đạt tăng tốc cực lớn ở Convolution và Mixed Layer (lên tới hơn 800 lần), và tăng tốc đáng kể ở Fully-Connected Layer (khoảng 50 lần).
- Ở mức model-level, thời gian suy luận giảm từ hàng tỷ chu kỳ xuống chỉ còn vài triệu chu kỳ, giúp hệ thống cải thiện khoảng 1226 lần về khả năng tính toán.
- Độ chính xác mô hình duy trì ở mức 98% trên tập MNIST, chứng minh rằng tăng tốc phần cứng không ảnh hưởng đến chất lượng kết quả.

3.5.2 So sánh với các nghiên cứu liên quan

Trong mục này, hệ thống FSPicoSoC được đối chiếu với AlPicoSoC của bài báo [?]. Cả hai cùng nhắm tới tăng tốc suy luận CNN trên PicoSoC có tích hợp bộ tăng tốc phần cứng. Tôi tách so sánh theo ba khía cạnh: tài nguyên và công suất, hiệu năng ở cấp độ model, và hiệu năng ở cấp độ layer.

A Tài nguyên & công suất

Tiêu chí	FSPicoSoC	AlPicoSoC [?]	So sánh (FSPicoSoC – AlPicoSoC)
FPGA	Zynq ZCU106	Ultra96-V2	—
Frequency (MHz)	100	100	—
LUTs	15,418	21,555	−6,137
FFs	6,391	11,940	−5,549
DSPs	162	-	-
BRAMs	180	90	+90
Dynamic Power (W)	0.190	0.106	+0.84
Device Static (W)	0.594	0.233	+0.361
Tổng công suất (W)	0.784	0.328	+0.456

Bảng 3.21: *So sánh tài nguyên, tần số và công suất giữa FSPicoSoC và AlPicoSoC.*

Nhận xét: Từ Bảng 3.21, FSPicoSoC sử dụng ít hơn 6,137 LUT và 5,549 FF so với AlPicoSoC, trong khi tăng thêm 162 DSP và 90 BRAM. Sự khác biệt này phù hợp với kiến trúc: (i) FSPicoSoC ánh xạ các phép nhân–cộng tích luỹ (MAC) xuống các DSP slices, giúp giảm đáng kể nhu cầu LUT/FF nhưng làm số DSP tăng; (ii) số lượng BRAM tăng do yêu cầu bộ nhớ IMEM/DMEM lớn hơn và bổ sung các buffer cho Accelerator; (iii) công suất tổng tăng thêm 0.456 W, chủ yếu đến từ công suất động của DSP và BRAM khi hoạt động song song. Nói cách khác, FSPicoSoC lựa chọn đánh đổi tài nguyên chuyên dụng (DSP, BRAM) để giảm áp lực lên tài nguyên logic tổng quát (LUT, FF) và đạt được thông lượng tính toán cao hơn.

B Hiệu năng ở cấp độ Layer (tham khảo).

Trong phần này, hiệu năng được phân tích ở cấp độ Convolution Layer, Mixed Layer và Fully-Connected Layer. Các số liệu trong mục này của ALPicoSoC chỉ mang tính *tham khảo*, do được đối chiếu và thu được trực tiếp từ bộ testcase được xây dựng trong luận văn, không phải từ dữ liệu chuẩn hoá của bài báo đối sánh.

Convolution Layer

Test Case	Chu kỳ			Lệnh thực thi		
	FSPicoSoC	AlPicoSoC	Chênh lệch	FSPicoSoC	AlPicoSoC	Chênh lệch
Testcase 0	879	2222	+1343	128	309	+181
Testcase 1	1761	3072	+1311	254	409	+155
Testcase 2	4057	5647	+1590	582	722	+140
Testcase 3	3077	5486	+2409	442	693	+251
Testcase 4	6262	8631	+2369	898	1063	+165
Testcase 5	14,949	18,120	+3171	2138	2188	+50
Testcase 6	101,049	136,249	+35,200	14,438	16,253	+1815

Bảng 3.22: *So sánh hiệu năng Convolution Layer giữa FSPicoSoC và AlPicoSoC (tham khảo).*

Nhận xét: Kết quả trong Bảng 3.22 cho thấy số chu kỳ và số lệnh thực thi của FSPicoSoC luôn thấp hơn so với AlPicoSoC. Chênh lệch số chu kỳ dao động từ khoảng +1,311 đến +35,200, trong khi số lệnh thực thi chênh lệch từ +50 đến +1,815. Điều này chứng tỏ việc tích hợp Flash Accelerator giúp tối ưu

hoá thời gian xử lý, đặc biệt rõ rệt ở các test case có cấu hình lớn (TC6). Tuy nhiên, mức cải thiện ở số lệnh thực thi nhìn chung thấp hơn so với số chu kỳ, cho thấy độ hiệu quả còn phụ thuộc vào đặc thù của từng cấu hình test case.

Mixed Layer

Test Case	Chu kỳ			Lệnh thực thi		
	FSPicoSoC	AlPicoSoC	Chênh lệch	FSPicoSoC	AlPicoSoC	Chênh lệch
Testcase 0	914	2,906	+1,992	134	406	+272
Testcase 1	1,754	3,356	+1,602	254	506	+252
Testcase 2	4,036	6,351	+2,315	580	827	+247
Testcase 3	3,406	6,851	+3,445	490	872	+382
Testcase 4	6,248	9,724	+3,476	896	1,226	+330
Testcase 5	14,935	19,927	+4,992	2,136	2,435	+299
Testcase 6	111,927	165,907	+53,980	15,992	19,354	+3,362

Bảng 3.23: *So sánh hiệu năng Mixed Layer giữa FSPicoSoC và AlPicoSoC (tham khảo).*

Nhận xét: AlPicoSoC không hỗ trợ trực tiếp *Mixed Layer*, do đó phải triển khai tuần tự: trước tiên thực hiện Convolution, sau đó cấu hình lại để tiếp tục Max Pooling. Ngược lại, FSPicoSoC đã tích hợp hai khối này thành một pipeline liền mạch, giúp giảm số chu kỳ từ hàng nghìn đến hàng chục nghìn so với AlPicoSoC (ví dụ Test Case 6 giảm 53,980 chu kỳ). Sự cải tiến này thể hiện ưu điểm nổi bật nhất của FSPicoSoC: giảm chi phí cấu hình lại phần cứng và duy trì dữ liệu trực tiếp trong pipeline, từ đó đạt hiệu năng vượt trội ở Mixed Layer.

Fully-Connected Layer

Test Case	Chu kỳ			Lệnh thực thi		
	FSPicoSoC	AlPicoSoC	Chênh lệch	FSPicoSoC	AlPicoSoC	Chênh lệch
Testcase 0	466	928	+462	70	132	+62
Testcase 1	1,166	1,609	+443	170	214	+44
Testcase 2	662	1,107	+445	98	154	+56
Testcase 3	1,712	2,168	+456	248	276	+28
Testcase 4	1,810	2,271	+461	262	290	+28
Testcase 5	4,946	5,367	+421	710	658	-52

Bảng 3.24: *So sánh hiệu năng Fully-Connected Layer giữa FSPicoSoC và AlPicoSoC (tham khảo).*

Nhận xét. Ở *Fully-Connected Layer*, kết quả cho thấy số chu kỳ của FSPicoSoC thấp hơn AlPicoSoC trong hầu hết các test case (giảm trung bình khoảng 430–460 chu kỳ). Về số lệnh thực thi, mức cải thiện nhỏ hơn so với Convolution và Mixed Layer, nhưng vẫn giữ xu hướng tối ưu. Đặc biệt, Test Case 5 cho thấy số lệnh của AlPicoSoC thấp hơn do cách tổ chức dữ liệu, tuy nhiên về tổng thể, FSPicoSoC vẫn duy trì được hiệu năng vượt trội và ổn định nhờ kiến trúc tăng tốc chuyên dụng cho các phép tính MAC.

C Hiệu năng ở cấp độ Model

Bảng 3.25 trình bày kết quả thực thi mô hình CNN: 2 Convolution + 2 Pooling + 2 Fully-Connected trên 10 mẫu MNIST. Kết quả được tách theo ba giai đoạn: Đọc dữ liệu, Tính toán, Ghi dữ liệu. Cột "Chênh lệch" được tính theo AlPicoSoC − FSPicoSoC (giá trị dương nghĩa là FSPicoSoC tiết kiệm hơn).

Nhận xét: FSPicoSoC tiết kiệm tổng cộng 4,218,484 chu kỳ và 453,620 lệnh so với AlPicoSoC khi thực thi 10 mẫu. Trong đó, tiết kiệm lớn nhất đến từ Đọc dữ liệu (giảm 2,093,745 chu kỳ) và Ghi dữ liệu (giảm 1,457,339 chu kỳ). Nguyên nhân là do trong AlPicoSoC, quá trình nạp dữ liệu vào IMEM/DMEM phải đi qua Processing System (PS), dẫn đến phát sinh độ trễ lớn khi giao tiếp qua AXI/UART. Ngược lại,

Giai đoạn	Chu kỳ			Lệnh thực thi		
	FSPicoSoC	**AlPicoSoC[?]**	**Chênh lệch**	**FSPicoSoC**	**AlPicoSoC[?]**	**Chênh lệch**
Đọc dữ liệu	333,050	2,426,795	2,093,745	65,160	217,390	152,230
Tính toán	5,934,550	6,601,950	667,400	842,060	1,132,830	290,770
Ghi dữ liệu	4,880,150	6,337,489	1,457,339	11,740	22,360	10,620
Tổng	**11,147,750**	**15,366,234**	**4,218,484**	**918,960**	**1,372,580**	**453,620**

Bảng 3.25: *So sánh hiệu năng theo giai đoạn của Model (2 Conv + 2 Pool + 2 Fully) giữa FSPicoSoC và AlPicoSoC trên 10 mẫu MNIST. Chênh lệch = AlPicoSoC − FSPicoSoC.*

FSPicoSoC chỉ đo độ trễ ở PL, dữ liệu được thao tác trực tiếp trong BRAM nội tại, nên số chu kỳ giảm mạnh. Ở tác vụ Tính toán, FSPicoSoC vẫn giảm được 667,400 chu kỳ nhờ phần cứng tăng tốc, cho thấy vai trò rõ rệt của Flash Accelerator. Đáng chú ý, lần này ngay cả phần lệnh của "Ghi dữ liệu" cũng giảm (ít hơn 10,620 lệnh), cho thấy FSPicoSoC xử lý khối I/O hiệu quả hơn.

Tính theo mỗi ảnh (vì bảng là 10 ảnh): - Chu kỳ/ảnh: FSPicoSoC $\approx$ 1,114,775 vs AlPicoSoC $\approx$ 1,536,623; chênh $\approx$ 421,848 chu kỳ/ảnh. - Ở 100 MHz: $421,848$ chu kỳ ≈ 4.22 ms tiết kiệm/ảnh. - Ở 10 MHz: $421,848$ chu kỳ ≈ 42.18 ms tiết kiệm/ảnh.

Mở rộng ra 10,000 ảnh MNIST: tổng chu kỳ tiết kiệm $\approx 4.22 \times 10^9$ chu kỳ. Tương ứng thời gian tiết kiệm ≈ 42.2 s ở 100 MHz, hoặc ≈ 421.8 s (~ 7.03 phút) ở 10 MHz. Kết quả này cho thấy lợi ích thực tế khi vận hành dài hạn: FSPicoSoC giảm đáng kể thời gian suy luận toàn bộ tập dữ liệu so với AlPicoSoC, trong khi phần tính toán vẫn là nguồn đóng góp chính cho mức tiết kiệm chu kỳ/lệnh.

Chương 4

Kết luận

4.1 Tổng kết

Trong luận văn này, tôi đã tiến hành thiết kế, triển khai và đánh giá một hệ thống FSPicoSoC – nền tảng PicoSoC được tích hợp thêm khối tăng tốc Flash Accelerator, hướng tới việc tăng tốc suy luận cho mạng nơ-ron tích chập (CNN) trên FPGA. Các kết quả nghiên cứu chính có thể được tổng kết theo ba hướng như sau:

- **Thiết kế kiến trúc hệ thống:**
  - FSPicoSoC được xây dựng trên lõi RISC-V PicoRV32, tích hợp khối tăng tốc FS_ACCEL thông qua bus 32-bit chung.
  - Các ngoại vi cơ bản gồm IMEM, DMEM, UART được duy trì, trong khi Accelerator đảm nhận các tác vụ tính toán chuyên dụng: nhân–cộng tích luỹ (MAC), lượng tử hoá (Quantize) và pooling.
  - Kiến trúc được tối ưu để hỗ trợ trực tiếp Mixed Layer (Convolution + Pooling) – điểm khác biệt quan trọng so với AlPicoSoC.
- **Phần mềm đi kèm:**
  - Xây dựng bộ firmware bằng ngôn ngữ C, hỗ trợ gọi các hàm cho Convolution, Fully-Connected và Mixed Layer.
  - Các lệnh đo đạc hiệu năng (rdcycle, rdinstreth) được tích hợp để đánh giá số chu kỳ và số lệnh thực thi.
  - Quy trình chuyển đổi từ mô hình huấn luyện sang các tham số (Input, Weight, Bias, Quantize) được tự động hoá để tạo testcase.
- **Triển khai và kiểm thử (trên ZCU106):**
  - Thiết kế được tổng hợp và hiện thực bằng Xilinx Vivado 2025.1, sinh bitstream và nạp lên Zynq UltraScale+ MPSoC ZCU106. Hệ thống vận hành ổn định ở 100 MHz, với xung 150 MHz trên board được Clocking Wizard chia xuống 100 MHz cho FSPicoSoC.
  - Chỉ IMEM được khởi tạo firmware bằng $readmemh (chứa mã lệnh điều khiển và tham số model: input/weight/bias/quant). Cách này cho phép hệ thống chạy hoàn toàn độc lập, không cần đến PS và không phát sinh độ trễ truyền dữ liệu từ PS sang PL. DMEM được khởi tạo rỗng để lưu kết quả trung gian và đầu ra.

  - Giao tiếp UART dùng để ghi log và đối soát kết quả suy luận; VIO (Virtual I/O) hỗ trợ khởi phát testcase và giám sát tín hiệu khi điều khiển board từ máy chủ (kết nối từ xa qua Tailscale).
  - Bộ testcase lớp (Conv/Mixed/Fully) và model (2 Conv + 2 Pool + 2 Fully) được chạy trên ZCU106; số chu kỳ và số lệnh thực thi được đo bằng rdcycle, rdinstret và ghi nhận qua UART để đối chiếu với mô phỏng.

Đặc tính chính của FSPicoSoC:

- Tập trung vào tăng tốc tính toán CNN – đặc biệt là phép MAC trong Convolution.
- Hỗ trợ trực tiếp Mixed Layer để giảm chi phí cấu hình lại giữa Convolution và Pooling.
- Giữ nguyên tính đơn giản của PicoSoC, dễ dàng triển khai trên các FPGA giá rẻ.
- Hiệu quả cả về tốc độ (hàng trăm lần tăng tốc) và điện năng (chỉ tăng nhẹ).

Hạn chế cần khắc phục trong tương lai:

- Hệ thống vẫn phụ thuộc vào UART để giao tiếp, khiến tốc độ đọc/ghi dữ liệu không được cải thiện đáng kể.
- Thiết kế chỉ tối ưu cho CNN đơn giản (MNIST), chưa thử nghiệm với các mô hình lớn hơn (CIFAR-10, ImageNet).
- Accelerator mới tập trung vào MAC + Quantize + Pooling, các tác vụ khác như BatchNorm, Skip Connection chưa được hỗ trợ.
- Tài nguyên BRAM chiếm nhiều (57,69%), hạn chế khả năng mở rộng.

Tóm lại, FSPicoSoC đã chứng minh được hiệu quả trong việc tăng tốc CNN trên FPGA, đạt được sự cân bằng tốt giữa hiệu năng, tài nguyên và độ chính xác. Đây là nền tảng khả thi để mở rộng sang các nghiên cứu sâu hơn về Deep Learning Accelerator trên thiết bị biên trong tương lai.

4.2 Tiềm năng phát triển

Hệ thống FSPicoSoC hiện tại đã chứng minh được khả năng tăng tốc đáng kể cho các lớp cơ bản của mạng nơ-ron tích chập (CNN). Trong tương lai, hệ thống có thể được mở rộng và phát triển theo một số hướng sau:

- **Mở rộng kích thước bộ nhân:** Hiện tại bộ nhân được thiết kế cho các hạt nhân tích chập 3×3. Trong tương lai, việc mở rộng lên 5×5 hoặc thậm chí lớn hơn sẽ giúp hệ thống có khả năng xử lý các mô hình CNN phức tạp hơn.
- **Hỗ trợ nhiều đặc trưng nâng cao của CNN:** FSPicoSoC có thể tích hợp thêm các khối chuyên dụng cho các tầng như *Batch Normalization*, *Residual Block* hay *Depthwise Separable Convolution*. Điều này sẽ cho phép hệ thống hỗ trợ các kiến trúc CNN hiện đại như ResNet, MobileNet hoặc EfficientNet.
- **Tối ưu hóa lượng tử hóa:** Ngoài chuẩn 8-bit hiện tại, hệ thống có thể mở rộng để hỗ trợ lượng tử hóa với độ chính xác thấp hơn, chẳng hạn như 4-bit hoặc 2-bit. Điều này giúp giảm thiểu tài nguyên phần cứng sử dụng, đồng thời cải thiện tốc độ xử lý và giảm tiêu thụ năng lượng, đặc biệt hữu ích trong các ứng dụng nhúng.